

计算机组成原理 第七版 习题答案

第一章	1
第二章	3
第三章	12
第四章	19
第五章	22
第七章	34
第八章	37
第九章	41
第十章	44

第一章

1. 模拟计算机的特点是数值由连续量来表示, 运算过程也是连续的。数字计算机的主要特点是按位运算, 并且不连续地跳动计算。模拟计算机用电压表示数据, 采用电压组合和测量值的计算方式, 盘上连线的控制方式, 而数字计算机用数字 0 和 1 表示数据, 采用数字计数的计算方式, 程序控制的控制方式。数字计算机与模拟计算机相比, 精度高, 数据存储量大, 逻辑判断能力强。

2. 数字计算机可分为专用计算机和通用计算机, 是根据计算机的效率、速度、价格、运行的经济性和适应性来划分的。

3. 科学计算、自动控制、测量和测试、信息处理、教育和卫生、家用电器、人工智能。

4. 主要设计思想是: 采用存储程序的方式, 编制好的程序和数据存放在同一存储器中, 计算机可以在无人干预的情况下自动完成逐条取出指令和执行指令的任务; 在机器内部, 指令和数据均以二进制码表示, 指令在存储器中按执行顺序存放。主要组成部分有: 运算器、逻辑器、存储器、输入设备和输出设备。

5. 存储器所有存储单元的总数称为存储器的存储容量。每个存储单元都有编号, 称为单元地址。如果某字代表要处理的数据, 称为数据字。如果某字为一条指令, 称为指令字。

6. 计算机硬件可直接执行的每一个基本的算术运算或逻辑运算操作称为一条指令, 而解算某一问题的一串指令序列, 称为程序。

7. 取指周期中从内存读出的信息流是指令流, 而在执行器周期中从内存读出的信息流是数据流。

8. 半导体存储器称为内存, 存储容量更大的磁盘存储器和光盘存储器称为外存, 内存和外存共同用来保存二进制数据。运算器和控制器合在一起称为中央处理器, 简称 CPU, 它用来控制计算机及进行算术逻辑运算。适配器是外围设备与主机联系的桥梁, 它的作用相当于一个转换器, 使主机和外围设备并行协调地工作。

9. 计算机的系统软件包括系统程序和应用程序。系统程序用来简化程序设计, 简化使用方法, 提高计算机的使用效率, 发挥和扩大计算机的功能用途; 应用程序是用户利用计算机来解决某些问题而编制的程序。

10. 在早期的计算机中, 人们是直接用机器语言来编写程序的, 这种程序称为手编程序或目的程序; 后来, 为了编写程序方便和提高使用效率, 人们使用汇编语言来编写程序, 称为汇编程序; 为了进一步实现程序自动化和便于程序交流, 使不熟悉具体计算机的人也能很方便地使用计算机, 人们又创造了算法语言, 用算法语言编写的程序称为源程序, 源程序通过编译系统产生编译程序, 也可通过解释系统进行解释执行; 随着计算机技术的日益发展, 人们又创造出操作系统; 随着计算机在信息处理、情报检索及各种管理系统中应用的发展, 要求大量处理某些数据, 建立和检索大量的表格, 于是产生了数据库管理系统。

11. 第一级是微程序设计级, 这是一个实在的硬件级, 它由机器硬件直接执行微指令; 第二级是一般机器级, 也称为机器语言级, 它由程序解释机器指令系统; 第三级是操作系统级, 它由操作系统实现; 第四级是汇编语言级, 它给程序人员提供一种符号形式语言, 以减少程序编写的复杂性; 第五级是高级语言级, 它是面向用户的, 为方便用户编写应用程序而设置的。用一系列的级来组成计算机的接口对于掌握计算机是如何组成的提供了一种好的结构和体制, 而且用这种分级的观点来设计计算机对保证产生一个良好的系统结构也是很有帮助的。

12. 因为任何操作可以由软件来实现, 也可以由硬件来实现; 任何指令的执行可以由硬件完

成,也可以由软件来完成。实现这种转化的媒介是软件与硬件的逻辑等价性。

13. 把运算器和控制器合在一起称为中央处理器,简称 CPU。其性能指标主要有以下几个方面:

(1)吞吐量:表示一台计算机在某一时间间隔内能够处理的信息量;

(2)响应时间:表示从输入有效到系统产生响应之间的时间间隔,用时间单位来度量;

(3)利用率:在给定的时间间隔内系统被实际使用的时间所占的比率,用百分比表示;

(4)处理机字长:指处理机运算器中一次能够完成二进制数运算的位数,如 32 位、64 位;

(5)总线宽度:一般指 CPU 中运算器与存储器之间进行互连的内部总线二进制位数;

(6)存储器容量:存储器中所有存储单元的总数目,通常用 KB、MB、GB、TB 来表示;

(7)存储器带宽:单位时间内从存储器读出的二进制数信息量,一般用字节数/秒表示;

(8)主频、时钟周期:CPU 的工作节拍受主时钟控制,主时钟是不断产生固定频率的时钟,主时钟的频率(f)称为 CPU 的主频,度量单位是 MHz(兆赫兹)、GHz(吉赫兹);主频的倒数称为 CPU 时钟周期(T), $T=1/f$,度量单位是 μs (微秒)、ns(纳秒);

(9)CPU 执行时间:表示 CPU 执行一般程序所占用的 CPU 时间,可用下式计算:CPU 执行时间=CPU 时钟周期数 $\times$ CPU 时钟周期

(10)CPI:表示每条指令周期数,即执行一条指令所需的平均时钟周期数,用下式计算:CPI=执行某段程序所需的 CPU 时钟周期数/程序包含的指令条数

(11)MIPS:Miion Instructions Per Second 的缩写,表示平均每秒执行多少百万条定点指令数,用下式计算:

$$\text{MIPS} = \text{指令数} / (\text{程序执行时间} \times 10)$$

(12)FLOPS:Floatimng-point Operations Per Second 的缩写,表示每秒执行浮点操作的次数,用来衡量机器浮点操作的性能。用下式计算:

$$\text{FLOPS} = \text{程序中的浮点操作次数} / \text{程序执行时间}(s)$$

14. 冯·诺依曼结构是一种将程序指令和数据存储在同一存储器中,并通过一套统一的总线系统(地址总线、数据总线和控制总线)进行访问的计算机体系结构。它的优势在于结构简单、实现成本低,适合早期的通用计算设备设计,而且程序可以像数据一样进行自修改操作。但由于程序和数据共用一条总线,导致 CPU 在取指令和读写数据之间必须交替进行,出现了所谓的“冯·诺依曼瓶颈”,严重限制了处理器性能的发挥。相较之下,哈佛结构将指令和数据分别存储在不同的存储器中,并配有各自独立的总线系统,允许 CPU 同时访问指令和数据,从而极大地提高了执行效率。这种结构特别适用于对实时性和速度要求较高的嵌入式系统和数字信号处理器(DSP),但其硬件复杂度和成本也相应增加,而且程序不能自修改,灵活性较差。为了解决两者的局限,改进的哈佛结构应运而生,它在硬件上仍保持指令和数据分离的设计,但允许在某些场合进行资源共享或交叉访问,比如引入统一的缓存系统或可共享的访问路径,从而在提高并行访问能力的同时保留了一定的结构灵活性。这种架构广泛应用于现代处理器中,如 ARM Cortex 系列,兼顾了冯·诺依曼结构的简洁与哈佛结构的高效,是对传统体系结构的一种综合优化。

15. 在评估计算机性能时,可以选择一些权威且广泛使用的基准测试工具。推荐如下几种:Geekbench、Cinebench R23、PassMark PerformanceTest、PCMark 10、3DMark。建议同学们根据自己的系统平台和使用需求,自行选择合适的工具进行测试,并记录相关参数和测试结果,作为对自己计算机性能了解的重要参考。

16. (1)平均 CPI=1.776 (2)MIPS=225.225 (3)执行时间: 575 μs

第二章

1. 根据原码、反码、补码之间的转换关系可得:

$$(1) [-35]_{\text{原}} = 1010\ 0011$$

$$[-35]_{\text{反}} = 1101\ 1100$$

$$[-35]_{\text{补}} = 1101\ 1101$$

$$(2) [-128]_{\text{补}} = 1000\ 0000$$

8 位二进制的原码和反码表示范围为-127~+127, 无法表示-128

$$(3) [-127]_{\text{原}} = 1111\ 1111$$

$$[-127]_{\text{反}} = 1000\ 0000$$

$$[-127]_{\text{补}} = 1000\ 0001$$

$$(4) [-1]_{\text{原}} = 1000\ 0001$$

$$[-1]_{\text{反}} = 1111\ 1110$$

$$[-1]_{\text{补}} = 1111\ 1111$$

2. 答: $[x]_{\text{补}} = a_7 a_6 a_5 \cdots a_0$

① 若 $a_7 = 0$, 则 $x \geq 0$, 满足 $x > -0.5$, 此时 a_0 到 a_6 可任意取值;

② 若 $a_7 = 1$, 则 $x < 0$, 要满足 $x > -0.5$, 需 $a_6 = 1$, 即 $a_7 = 1, a_6 = 1, a_0$ 到 a_5 不全为 0。

3. 字长 32 位浮点数, 阶码 8 位, 用移码表示, 尾数 23 位, 用补码表示, 基为 2

31	30	23	22	0
S	E (移码表示)		M (补码表示)	

由于 S 是数符, 已表示了尾数的符号, 所以为了提高表示精度, M (23 位) 不必存储符号位, 只需要存小数点后面的有效数值即可。

(1) 最大数的二进制表示为: 0 11111111 111111111111111111111111 (23 个 1);

(2) 最小数的二进制表示为: 1 11111111 000000000000000000000000 (23 个 0);

(3) 非 IEEE754 标准的补码表示的规格化数是指其最高位有效位与符号位相反, 故有:

① 最大正数为: 0 11111111 111111111111111111111111 (23 个 1) $= +(1-2^{-23}) \times 2^{127}$

② 最小正数为: 0 00000000 100000000000000000000000 (22 个 0) $= +2^{-1} \times 2^{-128}$

③ 最大负数为: 1 00000000 011111111111111111111111 (22 个 1) $= -(2^{-1} + 2^{-23}) \times 2^{-128}$

④ 最小负数为: 1 11111111 000000000000000000000000 (23 个 0) $= -1 \times 2^{127}$

所以其表示数的范围是: $-1 \times 2^{127} \sim -(2^{-1} + 2^{-23}) \times 2^{-128}$ 以及 $+2^{-1} \times 2^{-128} \sim +(1-2^{-23}) \times 2^{127}$ 。

4. 在 IEEE754 标准中,

$$(1) 27/64 = 0.011011 = 1.1011 \times 2^{-2}$$

$$E = -2 + 127 = 125 = 0111\ 1101 \quad S = 0 \quad M = 1011\ 0000\ 0000\ 0000\ 0000\ 000$$

最后表示为: 0 01111101 101100000000000000000000

$$(2) -27/64 = -0.011011 = -1.1011 \times 2^{-2}$$

$$E = -2 + 127 = 125 = 0111\ 1101 \quad S = 1 \quad M = 1011\ 0000\ 0000\ 0000\ 0000\ 000$$

最后表示为: 1 01111101 101100000000000000000000

5. (1) 用变形补码进行计算:

$$[x]_{\text{补}} = 00\ 11011 \quad [y]_{\text{补}} = 00\ 00011$$

$$\begin{array}{r} [x]_{\text{补}} = \quad 00\ 11011 \\ [y]_{\text{补}} = \quad + 00\ 00011 \\ \hline [x+y]_{\text{补}} = \quad 00\ 11110 \end{array}$$

结果没有溢出， $x+y=11110$

(2) $[x]_{\text{补}}=00\ 11011$ $[y]_{\text{补}}=11\ 01011$

$$\begin{array}{r} [x]_{\text{补}} = \quad 00\ 11011 \\ [y]_{\text{补}} = \quad + 11\ 01011 \\ \hline [x+y]_{\text{补}} = \quad 00\ 00110 \end{array}$$

结果没有溢出， $x+y=00110$

(3) $[x]_{\text{补}}=11\ 01010$ $[y]_{\text{补}}=11\ 11111$

$$\begin{array}{r} [x]_{\text{补}} = \quad 00\ 01010 \\ [y]_{\text{补}} = \quad + 00\ 11111 \\ \hline [x+y]_{\text{补}} = \quad 11\ 01001 \end{array}$$

结果没有溢出， $x+y=-10111$

6. $[x-y]_{\text{补}}=[x]_{\text{补}}+[-y]_{\text{补}}$

(1) $[x]_{\text{补}}=00\ 11011$ $[-y]_{\text{补}}=00\ 11111$

$$\begin{array}{r} [x]_{\text{补}} = \quad 00\ 11011 \\ [-y]_{\text{补}} = \quad + 00\ 11111 \\ \hline [x-y]_{\text{补}} = \quad 01\ 11010 \end{array}$$

结果有正溢出， $x-y=11010$

(2) $[x]_{\text{补}}=00\ 10111$ $[-y]_{\text{补}}=11\ 00101$

$$\begin{array}{r} [x]_{\text{补}} = \quad 00\ 10111 \\ [-y]_{\text{补}} = \quad + 11\ 00101 \\ \hline [x-y]_{\text{补}} = \quad 11\ 11100 \end{array}$$

结果没有溢出， $x-y=-00100$

(3) $[x]_{\text{补}}=00\ 11011$ $[-y]_{\text{补}}=00\ 10011$

$$\begin{array}{r} [x]_{\text{补}} = \quad 00\ 11011 \\ [-y]_{\text{补}} = \quad + 00\ 10011 \\ \hline [x-y]_{\text{补}} = \quad 01\ 01110 \end{array}$$

结果有正溢出， $x-y=10010$

7. (1) 用原码阵列乘法器：

$[x]_{\text{原}}=0\ 11011$ $[y]_{\text{原}}=1\ 11111$

因符号位单独考虑， $|x|=11011$ $|y|=11111$

$$\begin{array}{r} \\ \\ \hline \end{array}$$

1 1 0 1 1

```

      1  1  0  1  1
    1  1  0  1  1
  1  1  0  1  1
1  1  0  1  1
-----
1  1  0  1  0  0  0  1  0  1

```

$[x \times y]_{\text{原}} = 1\ 1101000101$

用补码阵列乘法器:

$[x]_{\text{补}} = 0\ 11011$ $[y]_{\text{补}} = 1\ 00001$

乘积符号位为: 1

$|x| = 11011$ $|y| = 11111$

```

      1  1  0  1  1
    ×) 1  1  1  1  1
    -----
      1  1  0  1  1
     1  1  0  1  1
    1  1  0  1  1
   1  1  0  1  1
  1  1  0  1  1
-----
1  1  0  1  0  0  0  1  0  1

```

$[x \times y]_{\text{补}} = 1\ 0010111011$

(2) 用原码阵列乘法器:

$[x]_{\text{原}} = 1\ 11111$ $[y]_{\text{原}} = 1\ 11011$

因符号位单独考虑, $|x| = 11111$ $|y| = 11011$

```

      1  1  1  1  1
    ×) 1  1  0  1  1
    -----
      1  1  1  1  1

```

```

      1  1  1  1  1
    0  0  0  0  0
  1  1  1  1  1
1  1  1  1  1
-----
1  1  0  1  0  0  0  1  0  1

```

$[x \times y]_{\text{原}} = 0\ 1101000101$

用补码阵列乘法器：

$[x]_{\text{补}} = 1\ 00001$ $[y]_{\text{补}} = 1\ 00101$

乘积符号位为：1

$|x| = 11111$ $|y| = 11011$

```

      1  1  1  1  1
    ×) 1  1  0  1  1
-----
      1  1  1  1  1
    1  1  1  1  1
  0  0  0  0  0
  1  1  1  1  1
  1  1  1  1  1
-----
1  1  0  1  0  0  0  1  0  1

```

$[x \times y]_{\text{补}} = 0\ 1101000101$

8. (1) $[x]_{\text{原}} = [x]_{\text{补}} = 0\ 11000$ $[-|y|]_{\text{补}} = 1\ 00001$

被除数 X $0\ 11000$

$+[-y]_{\text{补}}$ $1\ 00001$

余数为负 $1\ 11001 \rightarrow q_0 = 0$

左移 $1\ 10010$

$+ [y]_{\text{补}}$ $0\ 11111$

余数为正 $0\ 10001 \rightarrow q_1 = 1$

左移 1 00010
 $+[-y]补$ 1 00001

 余数为正 0 00011 $\rightarrow q_2=1$
 左移 0 00110
 $+[-y]补$ 1 00001

 余数为负 1 00111 $\rightarrow q_3=0$
 左移 0 01110
 $+[-y]补$ 0 11111

 余数为负 1 01101 $\rightarrow q_4=0$
 左移 0 11010
 $+[-y]补$ 0 11111

 余数为负 1 11001 $\rightarrow q_5=0$
 $+[-y]补$ 0 11111

 余数 0 11000

故 $[x \div y]原 = 1.11000$ 即 $x \div y = -0.11000$
 余数为 0.11000

(2) $[|x|]补 = 0\ 01011$ $[-|y|]补 = 1\ 00111$

被除数 X 0 01011
 $+[-y]补$ 1 00111

 余数为负 1 10010 $\rightarrow q_0=0$
 左移 1 00100
 $+[-y]补$ 0 11001

 余数为负 1 11101 $\rightarrow q_1=0$
 左移 1 11010
 $+[-y]补$ 0 11001

 余数为正 0 10011 $\rightarrow q_2=1$
 左移 1 00110
 $+[-y]补$ 1 00111

 余数为正 0 01101 $\rightarrow q_3=1$
 左移 0 11010
 $+[-y]补$ 1 00111

 余数为正 0 00001 $\rightarrow q_4=1$

左移 0 00010
+[-y]补 1 00111

余数为负 1 01001 $\rightarrow q_5=0$
+[y]补 0 11001

余数 0 00010

$x \div y = -0.01110$
余数为 0.00010

9. (1) $x = 2^{-011} * 0.100101, y = 2^{-010} * (-0.011110)$

$[x]_{\text{浮}} = 11101, 0.100101$

$[y]_{\text{浮}} = 11110, -0.011110$

$E_x - E_y = 11101 + 00010 = 11111$

$[x]_{\text{浮}} = 11110, 0.010010(1)$

$x+y$ 0 0.0 1 0 0 1 0 (1)
+ 1 1.1 0 0 0 1 0

1 1.1 1 0 1 0 0 (1)

规格化处理: 1.010010 阶码 11100

$x+y = 1.010010 * 2^{-4} = 2^{-4} * -0.101110$

$x-y$ 0 0.0 1 0 0 1 0 (1)
+ 0 0.0 1 1 1 1 0

0 0 1 1 0 0 0 0 (1)

规格化处理: 0.110000 阶码 11110

$x-y = 2^{-2} * 0.110001$

(2) $x = 2^{-101} * (-0.010110), y = 2^{-100} * 0.010110$

$[x]_{\text{浮}} = 11011, -0.010110$

$[y]_{\text{浮}} = 11100, 0.010110$

$E_x - E_y = 11011 + 00100 = 11111$

$[x]_{\text{浮}} = 11100, 1.110101(0)$

$x+y$ 1 1.1 1 0 1 0 1
+ 0 0.0 1 0 1 1 0

0 0.0 0 1 0 1 1

规格化处理: 0.101100 阶码 11010

$x+y = 0.101100 * 2^{-6}$

$x-y$ 1 1.1 1 0 1 0 1
+ 1 1.1 0 1 0 1 0

1 1 0 1 1 1 1 1

规格化处理: 1.011111 阶码 11100

$x-y = -0.100001 * 2^{-4}$

10. (1) $E_x = 0011, M_x = 0.110100$

$E_y = 0100, M_y = 0.100100$

$E_z = E_x + E_y = 0111$

$M_x * M_y$ 0.1 1 0 1
* 0.1 0 0 1

0 1 1 0 1

0 0 0 0 0

0 0 0 0 0

规格化: $2^6 \times 0.111011$

(2) $E_x = 1110$, $M_x = 0.011010$

$E_y = 0011$, $M_y = 0.111100$

$E_z = E_x - E_y = 1110 + 1101 = 1011$

$[M_x]_{\text{补}} = 00.011010$

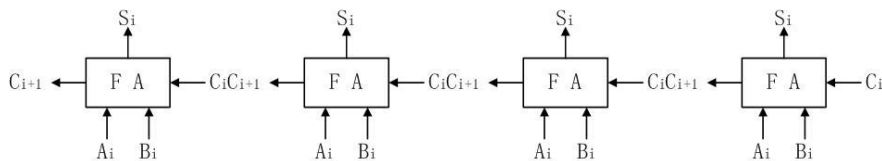
$[M_y]_{\text{补}} = 00.111100$, $[-M_y]_{\text{补}} = 11.000100$

```

      0 0 0 1 1 0 1 0
+[-My] 1 1 0 0 0 1 0 0
-----
      1 1 0 1 1 1 1 0      0
      1 0 1 1 1 1 0 0
+ [My] 0 0 1 1 1 1 0 0
-----
      1 1 1 1 1 0 0 0      0.0
      1 1 1 1 0 0 0 0
+ [My] 0 0 1 1 1 1 0 0
-----
      0 0 1 0 1 1 0 0      0.01
      0 1 0 1 1 0 0 0
+ [-My] 1 1 0 0 0 1 0 0
-----
      0 0 0 1 1 1 0 0      0.011
      0 0 1 1 1 0 0 0
+ [-My] 1 1 0 0 0 1 0 0
-----
      1 1 1 1 1 1 0 0      0.0110
      1 1 1 1 1 0 0 0
+ [My] 0 0 1 1 1 1 0 0
-----
      0 0 1 1 0 1 0 0      0.01101
      0 1 1 0 1 0 0 0
+ [-My] 1 1 0 0 0 1 0 0
-----
      0 0 1 0 1 1 0 0      0.01101
  
```

商 = 0.110110×2^{-6} , 余数 = 0.101100×2^{-6}

11.



4 位加法器如上图,

$$\begin{aligned}
 C_i &= A_i B_i + A_i C_{i-1} + B_i C_{i-1} \\
 &= A_i B_i + (A_i + B_i) C_{i-1} \\
 &= A_i B_i + (A_i \oplus B_i) C_{i-1}
 \end{aligned}$$

(1) 串行进位方式

$$\begin{array}{lll}
 C_1 = G_1 + P_1 C_0 & \text{其中: } G_1 = A_1 B_1 & P_1 = A_1 \oplus B_1 \text{ (} A_1 + B_1 \text{ 也对)} \\
 C_2 = G_2 + P_2 C_1 & G_2 = A_2 B_2 & P_2 = A_2 \oplus B_2 \\
 C_3 = G_3 + P_3 C_2 & G_3 = A_3 B_3 & P_3 = A_3 \oplus B_3 \\
 C_4 = G_4 + P_4 C_3 & G_4 = A_4 B_4 & P_4 = A_4 \oplus B_4
 \end{array}$$

(2) 并行进位方式

$$\begin{aligned}
 C_1 &= G_1 + P_1 C_0 \\
 C_2 &= G_2 + P_2 G_1 + P_2 P_1 C_0 \\
 C_3 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0 \\
 C_4 &= G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0
 \end{aligned}$$

12. IEEE 浮点格式的阶码减去 127 为真值指数。

(1) $-5 = -(101)_2 = -(1.01)_2 \times 2^2$, 故浮点格式为: 1 10000001 01000000000000000000

(2) $-1.5 = -(1.1)_2 = -(1.1)_2 \times 2^0$, 故浮点格式为: 1 01111111 100000000000000000000000

(3) $384 = (180)_{16} = (110000000)_2 = (1.1)_2 \times 2^8$, 故浮点格式为: 0 10000111 100000000000000000000000

(4) $1/16 = (1.0)_2 \times 2^{-4}$, 故浮点格式为: 0 01111011 000000000000000000000000

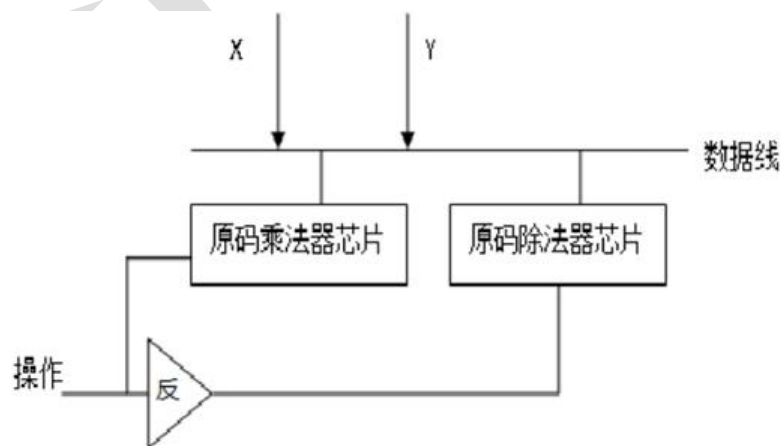
(5) $-1/32 = -(1.0)_2 \times 2^{-5}$, 故浮点格式为: 1 01111010 000000000000000000000000

13. (1) $(-1.11 \times 2^4)_2 = (-11100)_2 = (-28)_{10}$

(2) $(1.101 \times 2^{-1})_2 = (0.1101)_2 = (0.8125)_{10}$

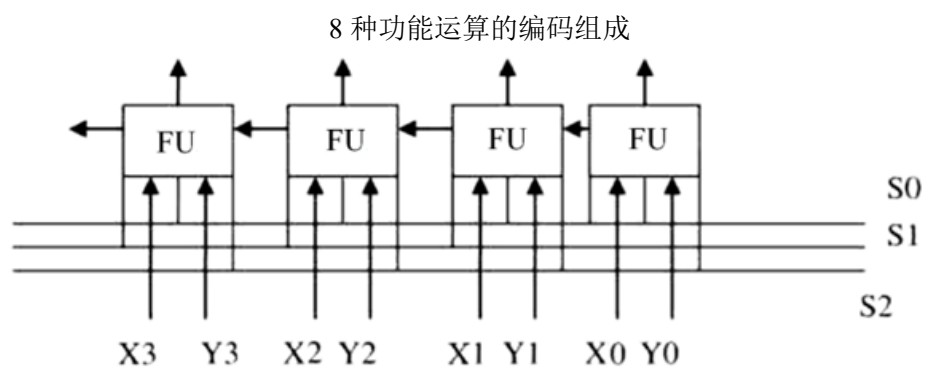
14. IEEE754 标准中, 32 位二进制数仍然有 2^{32} 种不同组合, 但是由于在 IEEE754 中, 阶码为全 1 且尾数为非 0 的情况不表示一个数。尾数 23 位, 尾数非 0 有 $2^{23}-1$ 种组合, 再配合符号位, 共有 $2 \times (2^{23}-1)$ 种组合不表示一个数, 所以该格式最多能表示不同的数的个数为 $2^{32} - 2 \times (2^{23}-1)$ 。

15. 设输入的数据为 X 和 Y, 则会把数据驱动到原码乘法器和原码除法器芯片上, 并且它是通过操作控制信号选通相应的芯片进行指定运算的, 因此设计如图所示:



16. 8 种功能运算, 需要采取的控制端个数为 3 个, 通过对控制端信号的逻辑组成产生相应运算功能的逻辑编码, 其对应关系如表所示。

功能	加	减	取反	取补	逻辑乘	逻辑加	传送	加 1
S0S1S2	000	001	010	011	100	101	110	111



完成 8 种功能的 ALU

第三章

1. (1) 20 位地址的存储器的存储空间为 2^{20} ，由于字长为 32 位，则存储容量为 $2^{20} \times 32 \text{ 位} = 4\text{M}$ 字节。

(2) 所需要的片数为 $\frac{1024\text{K} \times 32}{512\text{K} \times 8} = 2 \times 4 = 8$ 片

(3) SR

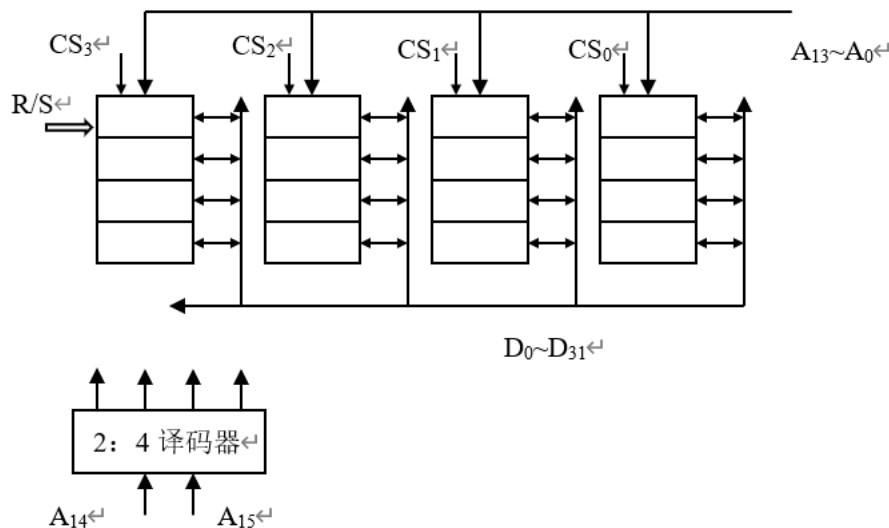
AM 芯片地址线位数为 19 根，存储器地址线 20 根，剩余一根地址线用于芯片选择。

2. (1) $2^{26}/2^{24} = 4$ (块)

(2) $(2^{24}/2^{22}) \times (64 \text{ 位}/8 \text{ 位}) = 32$ (片)

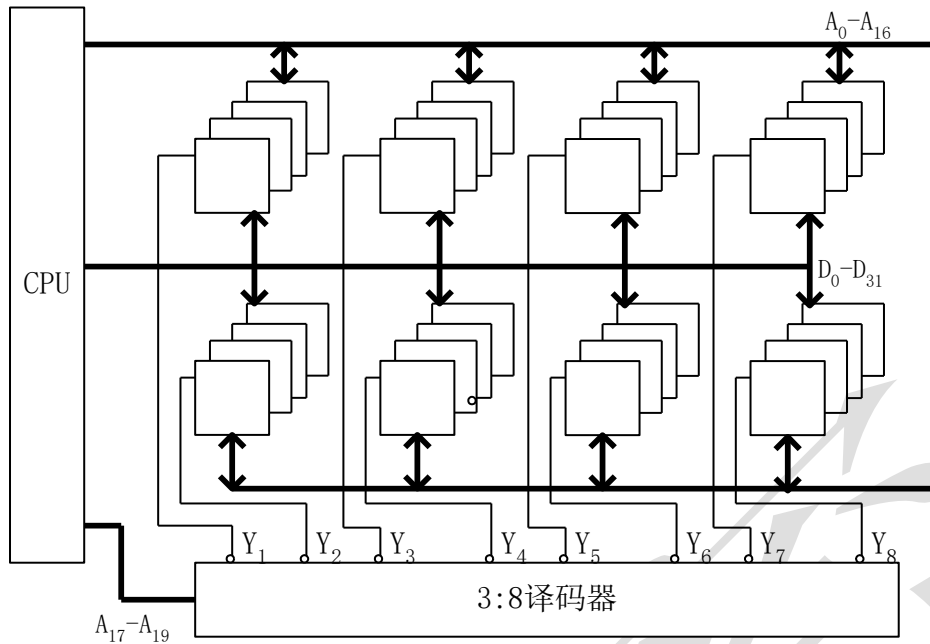
(3) 主存共需 DRAM 芯片为： $4 \times 32 = 128$ (片)，每个内存条有 32 片 DRAM 芯片，容量为 $16\text{M} \times 64$ 位，需 24 根地址线 ($A_{23} \sim A_0$) 完成内存条内存单元寻址。一共有 4 块内存条，采用 2 根高位地址线 ($A_{25} \sim A_{24}$)，通过 2-4 译码器译码产生片选信号对各模块板进行选择。

3. 根据题意，总存储容量为 $64\text{K} \times 32$ 位，单颗芯片容量为 $16\text{K} \times 8$ 位，则所需芯片数为 $(64\text{K}/16\text{K}) \times (32/8) = 16$ 颗。16 颗芯片通过两种方式组织：位扩展（增加字长）将 4 颗芯片分为一组，使数据位宽从 8 位扩展至 32 位。同一组内的芯片地址线并联，但数据线分别连接到数据总线的不同 8 位段上；字扩展（增加字数）需要 4 组这样的 $16\text{K} \times 32$ 位存储组，才能将总容量从 16K 扩展到 64K 。通过额外的地址线（高两位地址 A_{14} 、 A_{15} ）经过 2:4 译码器来选择具体哪一组工作。其组成逻辑图如图所示，其中使用一片 2:4 译码器。CPU 用高 2 位地址经 2:4 译码器译码后产生片选信号（CS）来选择 DRAM 芯片。

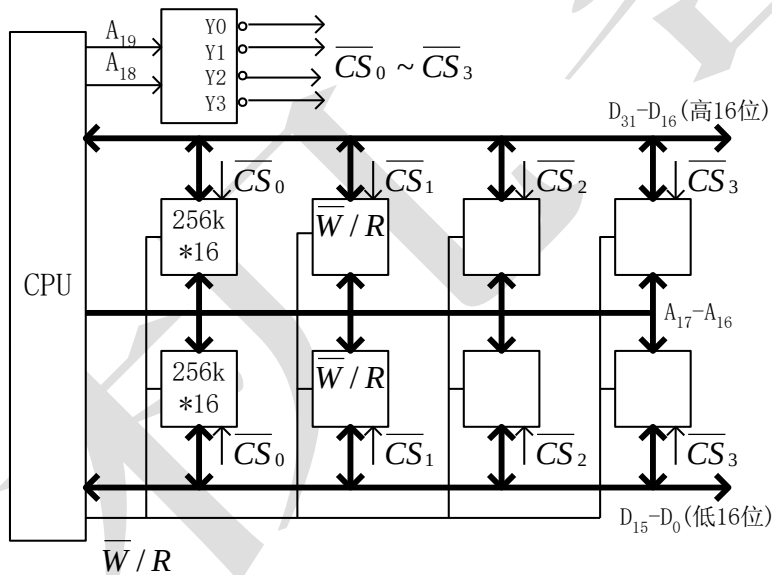


4. (1) $\frac{1024\text{K} \times 32}{128\text{K} \times 8} = 32$ 片

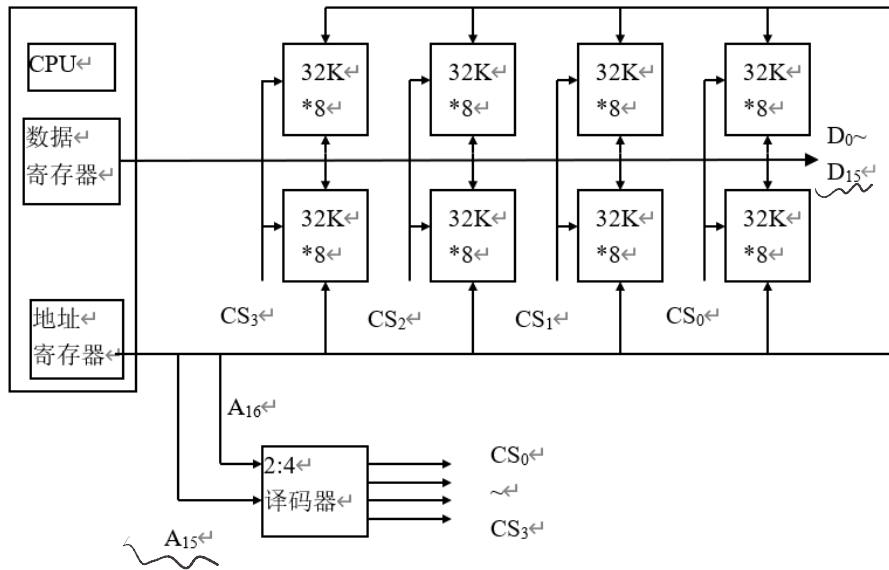
(2) DRAM 芯片的地址线为 17 根，1024K 的存储器的地址线为 20 根，因此需要字位同时扩展，4 片 DRAM 经字长位数扩展为 $128\text{K} \times 32$ 位的芯片，8 片这样的芯片经过地址串联为所需的存储器。于是可以将地址线中的高三位用作片选信号，经过 3:8 译码器译码后用于选择某一个芯片，如图所示。



5. 所设计的存储器单元数为 1M，字长为 32，故地址长度为 20 位 ($A_{19} \sim A_0$)，所用芯片存储单元数为 256K，字长为 16 位，故占用的地址长度为 18 位 ($A_{17} \sim A_0$)。由此可用位并联方式与地址串联方式相结合的方法组成整个存储器，共 8 片 RAM 芯片，并使用一片 2:4 译码器。其存储器结构如图所示。



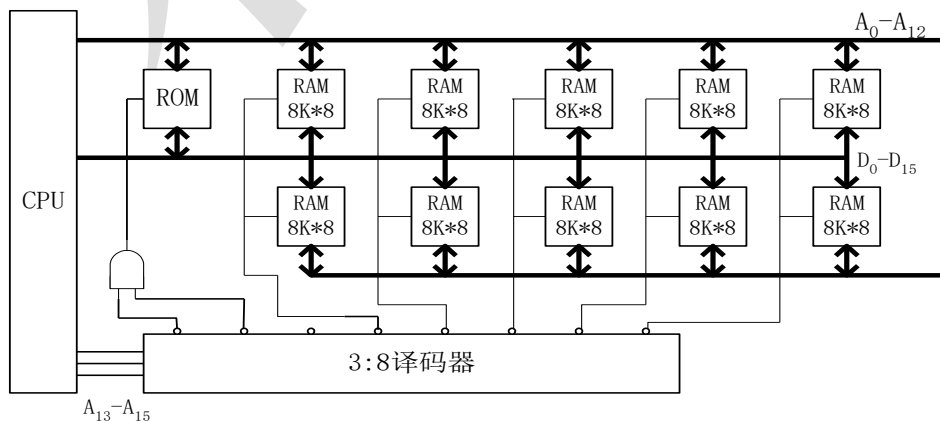
6. (1) 系统 16 位数据，所以数据寄存器 16 位
 (2) 系统地址 $128K = 2^{17}$ ，所以地址寄存器 17 位
 (3) 共需要 $128K \times 16 \text{ 位} / (32K \times 8 \text{ 位}) = 8$ 个
 (4) 芯片的字长为 8 位，地址线为 15 根，因此需要对芯片同时进行字位扩展。首先将 2 个 $E^2 \text{ PROM}$ 芯片经过字长扩展为 $32K \times 16$ 位的芯片，在此基础上，4 个 $32K \times 16$ 位的芯片经过地址串联为 $128K \times 16$ 位的只读存储器，同时地址线的高 2 位被 CPU 用作片选信号的输入，经过 2:4 译码器后产生片选信号，选定某一芯片。组成框图如图所示。



7. (1) ROM 区域占据了起始的 $16K \times 16$ 位的空间, 之后的 $8K \times 16$ 的空间空闲, 最后是 $40K \times 16$ 的 RAM 区域, 如图所示。

0000H	ROM
4000H	
6000H	RAM ₁
8000H	RAM ₂
A000	RAM ₃
C000H	RAM ₄
E000H	RAM ₅

(2) CPU 地址线为 16 根, RAM 芯片的地址线为 13 根, 因此芯片采用的片内地址为低 13 位, 即 $A_{12} \sim A_0$ 。高三位地址线用作片选信号的输入用于选片。需要的 RAM 芯片 ($8K \times 8$) 个数为 $40K \times 16 \text{ 位} / (8K \times 8) = 10$ 个。此时 2 个 RAM 芯片先经过字长扩展成字长为 16 位的芯片, 之后再经过地址串联为 $40K \times 16$ 位的 RAM 区域。译码器使用 3:8 译码器, 其中 $RAM_1 \sim RAM_5$ 各用两片 $8K \times 8$ 的芯片位并联连接, 如图所示。



8. 顺序存储器和交叉存储器连续读出 $m = 8$ 个字的信息总量都是:

$$q = 64 \text{ 位} \times 8 = 512 \text{ 位}$$

顺序存储器和交叉存储器连续读出 8 个字所需的时间分别是:

$$t_1 = mT = 8 \times 100\text{ns} = 8 \times 10^{-7}\text{s}$$

$$t_2 = T + (m - 1)\tau = 100\text{ns} + 7 \times 50\text{ns} = 450\text{ns} = 4.5 \times 10^{-7}\text{s}$$

顺序存储器和交叉存储器的带宽分别是:

$$W_1 = q/t_1 = 512 \div (8 \times 10^{-7}) = 64 \times 10^7 [\text{b/s}]$$

$$W_2 = q/t_2 = 512 \div (4.5 \times 10^{-7}) = 113.8 \times 10^7 [\text{b/s}]$$

9. cache 的命中率

$$H = \frac{N_c}{N_c + N_m} = \frac{2420}{2420 + 80} = 0.968$$

主存慢于 cache 的倍率为

$$r = \frac{T_m}{T_c} = \frac{240}{40} = 6$$

cache/主存系统效率 e 为

$$e = \frac{1}{r + (1 - r)H} * 100\% = \frac{1}{6 + (1 - 6) * 0.968} * 100\% = 86.2\%$$

平均访问时间 T_a 为

$$T_a = \frac{T_c}{e} = \frac{40\text{ns}}{0.862} = 46.4\text{ns}$$

10. 已知 cache/主存系统平均访问时间 $t_a = 50\text{ns}$, 而 $t_a = h \times t_c + (1 - h) \times t_m$, 所以有 $h = (t_a - t_m)/(t_c - t_m) = (50 - 200)/(40 - 200) = 150/160 = 93.75\%$ 。

11. 设取指周期为 T , 总线传送周期为 τ , 指令执行时间为 t_0

$$(1) t = (T + 5\tau + 6t_0) \times 80 = 80T + 400\tau + 480t_0$$

$$(2) t = (T + 7\tau + 8t_0) \times 60 = 60T + 420\tau + 480t_0$$

故程序运行时间不相等。

12. 在二级存储系统中满足 $T_a = H \times T_1 + (1 - H) \times T_2$ 。

13. 主存储器共包含 $4\text{K} \times 128 = 512\text{K} = 2^{19}$ 字, 所以主存地址长度为 19 位。每块有 $128 = 2^7$ 字, 所以块内字需要 7 位表示。cache 由 64 个行组成, 每组 4 行, 共分成 16 组, 所以组号需要 4 位表示。剩余的 $19 - 4 - 7 = 8$ 位用作标记。所以, 主存地址格式为:

Tag (8 位)	组号 (4 位)	字 (7 位)
-----------	----------	---------

14. 内存中共有 $1\text{MB}/(1 \times 16)\text{B} = 64\text{K}$ 个块。cache 中可以存放的内存块数为 $64\text{KB}/16\text{B} = 4\text{K}$ 个, 即有 4K 行。一块有 $16\text{B}/1\text{B} = 16$ 个字。所以 cache 的行号 i 和主存块号 j 有如下的函数关系: $i = j \bmod m$, 其中 $m = 4\text{K} = 4096$ 。所以内存中映射到第一行的是第 0 块, 第 m 块, 第 $2m$ 块, ...。即第 0 块, 第 4096 块将映射到同一行, 它们的内存地址为 00000H 和 $4096 \times 16 = 10000\text{H}$ 。

15. 主存中的块数为 $16\text{M} \times 32\text{位}/(4 \times 32)\text{位} = 4\text{M}$, cache 中的块数为 $64\text{K} \times 32\text{位}/4 \times 32\text{位} = 16\text{K}$ 。所以 cache 的行号 i 和主存块号 j 有如下的函数关系: $i = j \bmod m$, 其中 $m = 16\text{K}$ 。主存地址单元格数为 $16\text{M} = 2^{24}$, 于是主存地址长度为 24 位; 每块 $4 \times 32\text{位}$, 包含 4 个字, 于是块内字用 2 位表示; cache 行数为 $16\text{K} = 2^{14}$, 于是行号用 14 位表示; 最后, 标记位数为 $24 - 2 - 14 = 8$ 。主存地址格式为:

Tag (8 位)	行号 (14 位)	字 (2 位)
-----------	-----------	---------

16. BD

【解析】多级存储器由高速缓冲存储器 cache、主存储器和辅助存储器，由这三者构成一个统一的存储系统。在虚拟存储器中，外存和主存的工作方式不同。

17. A

【解析】多道程序设计的主要优点是通过将用户的 CPU 请求和 I/O 请求重叠起来的办法来有效地使用 CPU 和外设。它设法使 CPU 总处于工作状态，以此来提高 CPU 和外设的利用率。

18. D

【解析】段页式的优点是便于动态申请内存空间、利用率高、管理和使用统一化、便于共享和保护、便于动态链接；缺点是有碎片问题。

19.6

【解析】LRU 算法的思想：每页设置一个计数器，每次命中一页，该页对应的计数器清零，其他各页的计数器加 1；需要替换时，将计数值最大的页换出，所以，对应的访问过程及相应的计数器的内容，替换结果如图所示。其中括号标注的页是未命中的页，即失效页面，共 6 次。

访问序列		1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3	7
调入的页号	a	(1)	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	b		(8)	8	8	8	8	8	8	8	8	8	8	8	8	8	8	(7)	7	7	7
	c				(7)	7	7	7	7	7	7	(3)	3	3	3	3	3	3	3	3	3
	d					(2)	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
各计数器的值	a	0	1	0	1	2	3	4	5	0	1	2	3	4	0	1	0	1	0	1	2
	b		0	1	2	0	1	2	3	4	0	1	0	1	2	3	4	0	1	2	0
	c				0	1	2	0	1	2	3	0	1	2	3	0	1	2	3	0	1
	d					0	1	0	1	2	3	4	0	1	2	3	4	5	6	7	

20. 主存容量为 $4\text{MB}=2^{22}$, 故物理地址为 22 位; 虚存容量为 $1\text{GB}=2^{30}$, 故虚拟地址为 30 位; 页表长度, 即页面数 $=1\text{GB}/4\text{KB}=2^{18}=256\text{K}$ 。
21. (1) 若页表存放在主存中, 则要实现一次主存访问需两次访问主存: 一次是访问页表, 确定所存取页面的物理地址; 第二次才根据该地址存取页面数据。故访问一次主存的时间为 $50 \times 2 = 100\text{ns}$ 。
- (2) $75\% \times 50 + (1 - 75\%) \times 2 \times 50 = 62.5\text{ns}$
22. 被访问的字在 cache 中的概率为 0.9;
不在 cache 但在主存中的概率为 $(1-0.9) \times 0.6 = 0.06$;
不在 cache 也不在主存中的概率为 $(1-0.9) \times (1-0.6) = 0.04$;
所以, 一个字的访问时间为:
 $15 \times 0.9 + (15+60) \times 0.06 + (15+60+10 \times 10^6) \times 0.04 = 13.5 + 4.5 + 400003 = 400021\text{ns}$
23. 逻辑地址空间包含 16 页, 页大小为 2KB, 所以逻辑地址空间为 32KB, 逻辑地址应用 $4+11=15$ 位, 物理空间共有 8 页, 页大小为 2KB, 所以主存物理空间 16KB。
24. 页长 1KB, 所以页内地址为 10 位。主存物理页面数为 16 页, 共 14 位地址码 (其中页面号 4 位, 页内地址 10 位), 用户虚地址空间为 32 页, 页面号为 0~31; 共 15 位地址码 (其中页面号 5 位, 页内地址 10 位)。
- 0AC5H=00010,1011000101B, 页面号为 2, 已被调入到主存页 4, 所以, 物理地址中的页面号为 4, 页内地址与虚地址的页内地址相同, 所以是: 0100,1011000101=12C5H;
1AC5H=00110,1011000101B, 页面号为 6, 未被调入到主存页中, 所以无物理地址, 会发生缺页中断。
25. 虚拟管理是由软件 (操作系统) 和硬件共同完成, 由于软件的介入, 虚存对实现存储管理系统程序不透明。而段是按照程序的自然分界划分的长度可以动态改变的区域。通常, 程序员把子程序、操作数和常数等不同类型的数据划分到不同的段中, 并且每个程序可以有多个相同类型的段。由于分段是由程序员完成的, 所以段式虚拟存储器对程序员而言不是透明的, 但虚存到实存的地址映射是由系统软件辅助完成的, 故对应用程序而言, 段式虚拟存储器是“半透明”的。
26. 在有虚拟存储管理的系统中, 程序不是一次性装入内存才运行, 所以不是所有页面都必须处在主存中, 而是根据程序的局部性, 有的页面在主存, 有的页面在辅存。
27. 由于物理页与虚拟页的页面大小相同, 且为 2 的整数次幂, 所以页式虚拟存储器地址变换时可以用物理页号与页内偏移量直接拼接成物理地址。而段式虚拟存储器的各段大小不同, 且段起始地址任意, 所以必须用段起址与段内偏移量相加才能得到物理地址。
28. 产生颠簸的原因主要有: 分配的页面数太少、替换策略不佳; 防止颠簸的办法: 适当增加分配给用户程序的页面数、选取 LRU 或更好的替换策略。
29. (1) 小端模式: 0x6000 存放 0x34, 0x6001 存放 0x12。
(2) 大端模式: 0x6000 存放 0x12, 0x6001 存放 0x34。
- 30.
- (1) DDR4 内存子系统
作为主存系统的基础, 包含 8 通道 DDR4 内存控制器, 每个通道最大支持 1TB 容量, 总带宽可达 1.5Tb/s。DDR4 内存直接与处理器内核连接, 提供大容量存储空间, 支持单时钟周期访问。
- (2) 三级缓存 (L1/L2/L3 Cache)
- (2.1) L1 缓存: 每个 CPU 核心独享 64KB 指令缓存 (L1I) 和 64KB 数据缓存 (L1D), 访问延时仅 1 个时钟周期。

(2.2)L2 缓存：每个核心独享 512KB 共享缓存，访问延时 15 个时钟周期。

(2.3)L3 缓存：64MB 共享缓存，分为 Tag 和 Data 两部分，Tag 负责地址映射，Data 存储实际数据，访问延时 3 个时钟周期。该设计通过共享机制提升多核间数据交换效率。

(3)内存控制器与互联架构

集成 8 通道 DDR4 控制器，支持高速数据传输（最高速率 2.6GHz）。通过华为自研的 HCCS 总线实现片间互联，支持四颗处理器互联形成 NUMA 架构，扩展至 256 核。同时，Hydra 根代理模块保障多芯片系统的缓存一致性。

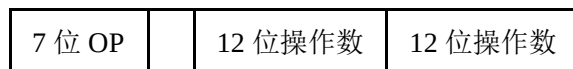
这一分层结构通过减少内存访问延迟（如 L1 缓存单周期访问）和优化带宽（如 DDR4 多通道设计），显著提升了高性能计算和数据处理场景的效率。

第四章

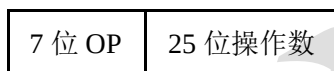
1. 不合理。一个字符的 ASCII 是 7 位，如果设计主存单元字长为 32 位，则一个单元可以放四个字符，这是可行的。指令最好是半字长或单字长，若设计成 12 位，则不利于指令的存储和读取，应该设计 16 位比较合适。

2. 指令系统共有 70 条指令，故操作码字段需要至少 7 位。

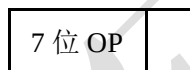
双操作数指令格式：



单操作数指令格式：



无操作数指令格式：



3. 该指令格式及寻址方式特点有：①单字长二地址指令；②操作码字段 OP 有 6 位，故可指定 $2^6=64$ 条指令；③源和目标操作数都存在通用寄存器中，可分别指向 16 个寄存器，所以是 RR 型指令；④这种指令结构常用于寄存器间的数据传送及算术逻辑运算类指令。

4. 该指令格式及寻址方式特点如下：①双字长二地址指令；②操作码字段 OP 有 6 位，故可指定 $2^6=64$ 条指令；③一个操作数在通用寄存器（可指向 16 个）中，另一个操作数在主存中，故为 RS 型指令；④第二个操作数的有效地址可通过变址寻址得到，即变址寄存器（共 16 个）内容加上偏移量。

5. 该指令格式及寻址方式特点如下：①双操作数指令；②两个操作数地址均由 8 种寻址方式和 8 个寄存器构成；③操作码字段 OP 有 4 位，故可指定 $2^4=16$ 条指令。

6. (1) 直接寻址方式

(2) 相对寻址方式

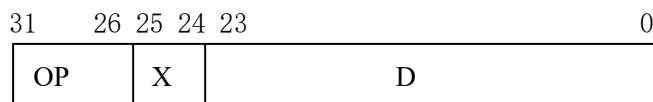
(3) 变址寻址方式

(4) 基址寻址方式

(5) 间接寻址方式

(6) 基址间接寻址方式

7. 40 条指令至少需要操作码字段 6 位，所以剩下的长度为 26 位。主存的容量为 64M 字，四种寻址方式则设寻址模式 (X) 2 位，格式如下：



X=00，直接寻址，有效地址 E=D

X=01，立即寻址，D 字段为立即数

X=10，变址寻址，有效地址 E=(RX)+D （可寻址 64M 个存储单元）

X=11，相对寻址，有效地址 E=(PC)+D （可寻址 64M 个存储单元）

其中 RX 为变址寄存器（32 位），PC 为程序计数器（32 位）。在相对寻址时，位移量 D

可正可负。

8.(1)50 种操作码占 6 位, 4 种寻址方式占 2 位。以单地址指令为例, 用 24 位表示操作数字:

OP (6)	X (2)	D (24)
--------	-------	--------

X = 00, 寄存器寻址方式。D 字段实际使用 4 比特选择 16 个通用寄存器。

X = 01, 寄存器间接寻址方式。D 字段实际使用 4 比特选择 16 个通用寄存器。E = (RX)。

X = 10, 立即寻址方式。D 字段给出 24 位立即数。

X = 11, 直接寻址方式。D 字段给出 24 位内存地址。E = D。

(2) 寻址模式字段变成 3 位, 可以支持更多的寻址方式。可增加相对寻址方式, 其有效地址 $E = PC + D$; 还可使用内存间接寻址, 此时有效地址 $E = (D)$ 。

9. 16 个通用寄存器占 4 位, 64 种操作占 6 位, 剩下 22 位用于存储器地址,

OP (6)	R (4)	D (22)
--------	-------	--------

采用 R 为基址寄存器寻址, 地址 = (R) + D

当基址最大, D 也是最大的时候, 寻址能力最大

而寄存器是 32 位的,

故最大存储空间是 $2^{32} + 2^{22} = 4GB + 4MB$ 。

10. 表 4.7 的指令数为 29, 则指令的操作码至少为 5 位。设这些指令支持立即寻址、寄存器寻址、直接寻址、堆栈寻址、相对寻址、内存间接寻址、寄存器间接寻址、变址寻址、基址寻址等 9 种寻址方式。并设计算机字长为 32 位:

6	4	8	4	8
OP	源寻址方式	源操作数	目标寻址方式	目标操作数

11.C

【解析】A 项, RISC 指令系统的控制器采用组合逻辑控制, 计算机体系结构更复杂; B 项, RISC 指令系统是在 CISC 指令系统上的改进, 精简了指令集, CPU 中增加了通用寄存器, 采用了流水线技术, 优化了编译程序; D 项, RISC 中像乘、除法和浮点数运算等复杂指令的功能由频度高的简单指令的组合来实现。

12.(1)寄存器

(2)寄存器间接

(3)立即

(4)直接

(5)相对寻址、基址、变址

13. 临时变量 $t0 = g + h$ 用 \$t0 代替, 临时变量 $t1 = i + j$ 用 \$t1 代替, 则 MIPS R4000 汇编语言表示如下:

add \$t0,\$s1,\$s2 #完成 $t0 = g + h$

add \$t1,\$s3,\$s4 #完成 $t1 = i + j$

sub \$s0,\$t0,\$t1 #完成 $f = t0 - t1$

14. 由于 MIPS 系统的寄存器 \$s0~\$s7 对应的寄存器号为 16~23 (十进制), 寄存器 \$t0~\$t7 对应的寄存器号为 8~15。再结合 MIPS 指令字段可写出十进制数表示的 3 条机器语言指令如下表所示。

指令	机器语言各字段					
各字段长度	6 位	5 位	5 位	5 位	5 位	6 位
R 型	op	rs	rt	rd	shamt	funct
I 型	op	rs	rt	address		
lw \$t0,1200(\$t1)	35	9	8	1200		
add \$t0,\$s2,\$t0	0	18	8	8	0	32
sw \$t0,1200(\$t1)	43	9	8	1200		

15. ARM 是 32 位计算机, 按其指令机器格式, 十进制和二进制表示的机器语言如下表所示。

	cond	F	opcode			Rn	Rd	offset 12
			I	opcode	S			operand 12
十进制	14	0	0	4	0	1	5	2
二进制	1110	00	0	0100	0	0001	0101	0000 0000 0010

16. 设 $g+h$ 的求和结果暂存在寄存器 $r5$, $i+j$ 的求和结果暂存在寄存器 $r6$, 则 ARM 汇编语言代码如下所示:

ADD r5,r1,r2 #r5=r1+r2=g+h

ADD r6,r3,r4 #r6=r3+r4=i+j

SUB r0,r5,r6 #r0=r5-r6=f-(g+h)-(i+j)

17. 指令系统的操作码采用分层扩展编码, 通过操作码长度的动态变化实现不同地址数指令的兼容, 依次涉及三地址指令、二地址指令、单地址指令和零地址指令。

(1) 三地址指令: OP 为 4 位, 可支持 16 种指令(0000~1111), 需支持 11 条三地址指令, 占用操作码 0000~1010, 剩下 5 种操作码(1011~1111)用于后续拓展。

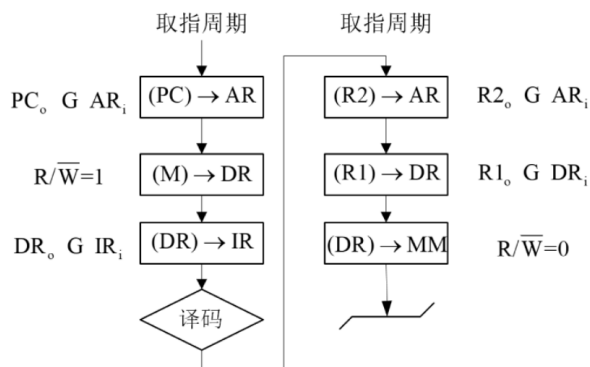
(2) 二地址指令: OP 为 8 位, 三地址剩余 5 种操作码(1011~1111)作为前缀, 后接 4 位扩展码, 每前缀支持 16 条指令(共 $5 \times 16 = 80$ 条)。需支持 70 条二地址指令, 占用其中 70 种组合(如 1011 0000~1110 1101), 剩余 10 种未用。

(3) 单地址指令: 二地址未用的 10 种操作码作为前缀(前 8 位), 后接 4 位扩展码, 每前缀支持 16 条指令(共 $10 \times 16 = 160$ 条)。需支持 150 条单地址指令, 占用其中 150 种组合(如 1111 0110 0000~1111 1111 1111), 剩余 10 种未用。

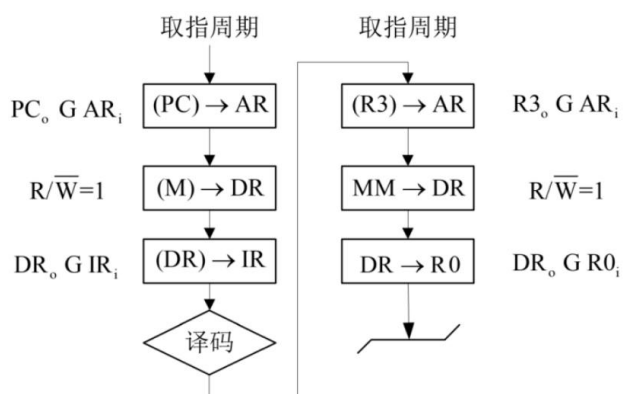
因此, 对于零地址指令, 剩下的指令最大数量, OP 占用 16 位。零地址指令使用所有未被三地址、二地址、单地址指令占用的操作码组合。根据分层扩展原理, 剩余操作码空间为单地址未用的 10 种前缀(前 12 位), 每前缀后接 4 位扩展码, 支持 $10 \times 16 = 160$ 条零地址指令。

第五章

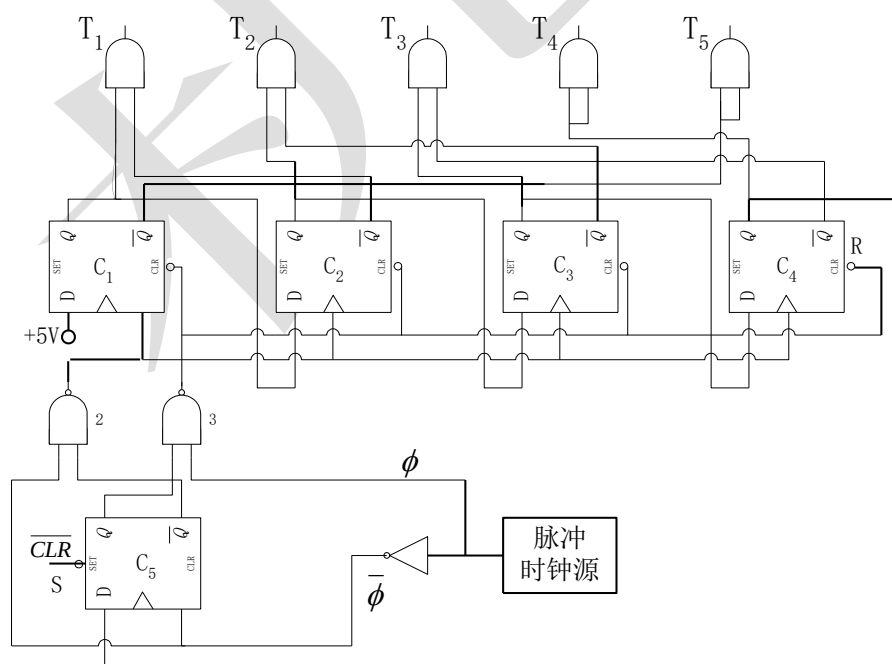
1. (1) IR、(2)PC、(3)DR、通用寄存器
- 2.



- 3.



- 4.



- 5.

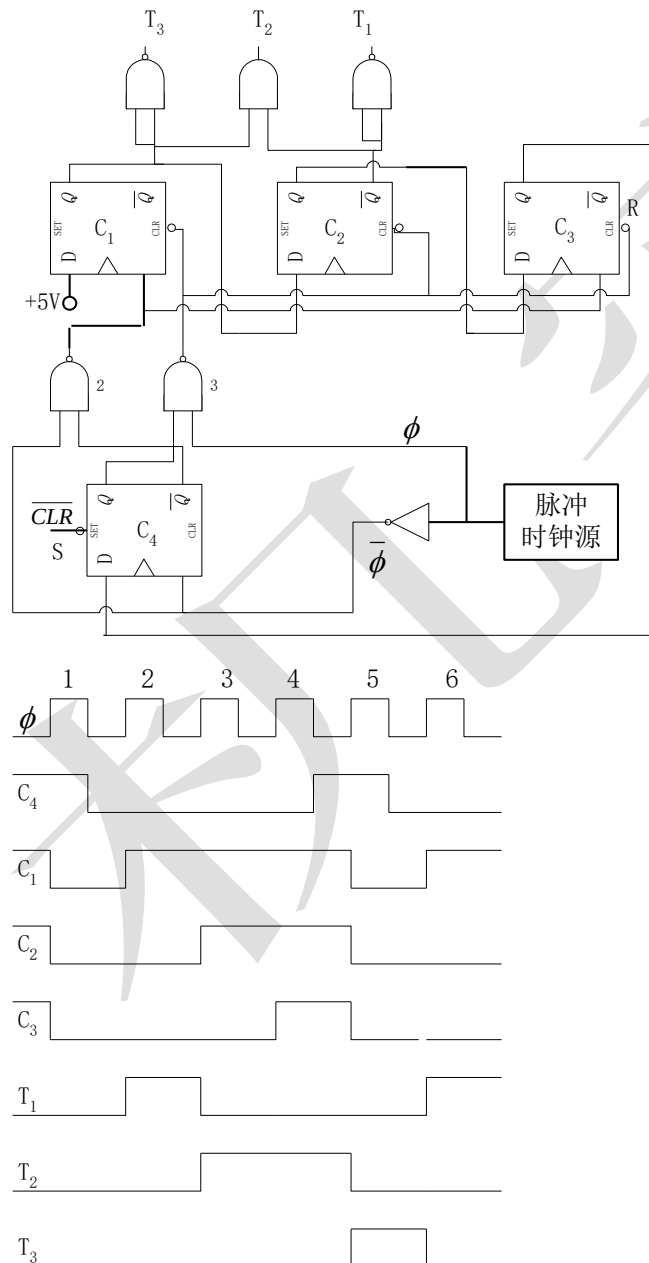
节拍脉冲 T_1, T_2, T_3 的宽度实际上等于时钟脉冲的周期或是它的倍数。此处 $T_1 = T_2 = 200\text{ns}$,

$T_3 = 400\text{ns}$, 所以主脉冲源的频率应为 $f = \frac{1}{T} = 5\text{MHz}$ 。

为了消除节拍脉冲上的毛刺, 环形脉冲发生器采用移位寄存器形式。图中画出了题目要求的逻辑电路图与时序信号关系图。根据时序信号关系, T_1, T_2, T_3 三个节拍脉冲的逻辑表达式如下:

$$T_1 = C_1 * \overline{C_2} \quad T_2 = C_2 \quad T_3 = \overline{T_1}$$

T_1 用与门实现, T_2 和 T_3 则用 C_2 的 $\overline{Q}$ 端和 C_1 的 Q 端加非门实现, 其目的在于保持信号输出时延时间的一致性并与环形脉冲发生器隔离。



6. 平均每条指令由 4 条微指令组成, 其中一条为公用的, 则该机器共有 $80 \times (4-1) + 1 = 241$

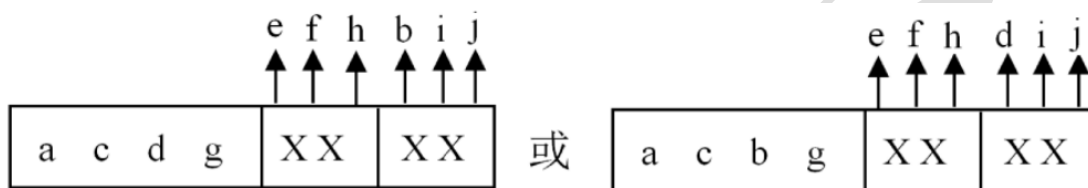
条微指令。

微指令长度 32 位，所以控制存储器容量约为 $241 \times 32 \text{bit} = 964 \text{B}$ 。

7.

$$M = G$$
$$S3 = H + D + F$$
$$S2 = 1$$
$$S1 = A + B + F + G$$
$$C = I + D + Ey + Fy + G\varphi$$

8. 首先要从表中找出相斥类的微命令组, 进行分组译码。经分析, (e, f, h) 和 (b, i, j)、(d, i, j) 和 (e, f, h) 均是不可能同时出现的互斥信号, 故可将其通过 2: 4 译码后输出三个微命令信号 (00 不用), 其余四个微指令控制信号可直接控制。微指令的控制字段格式如图所示。



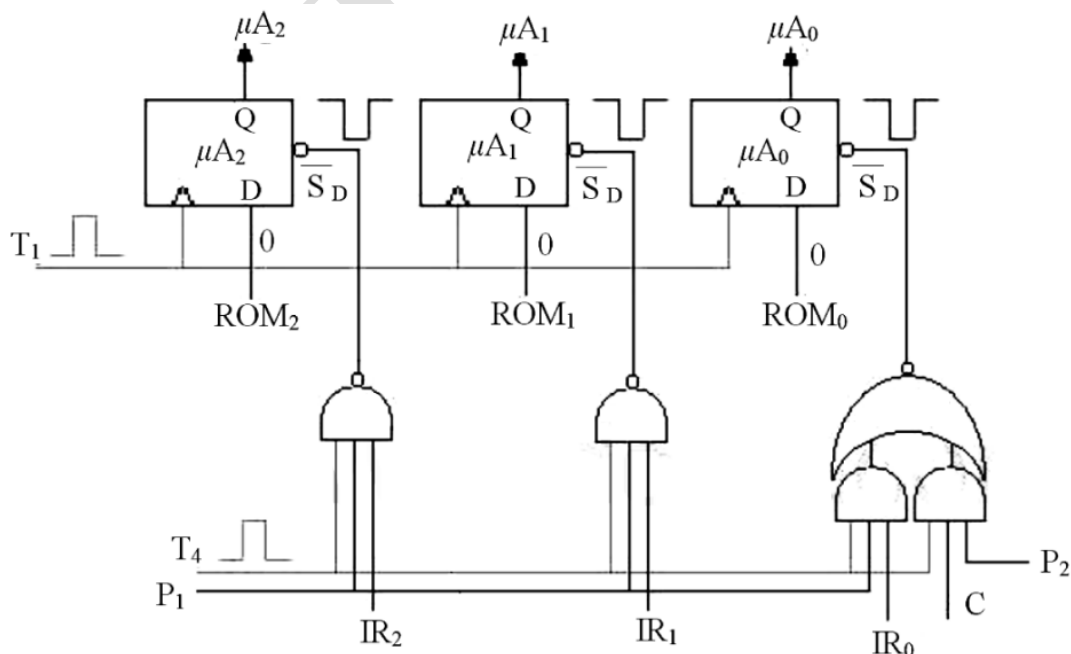
9.转移逻辑表达式:

$$\mu A_5 = P_3 \cdot IR_5 \cdot T_4 \quad \mu A_4 = P_3 \cdot IR_4 \cdot T_4$$

$$\mu A_3 = P_1 \cdot IR_3 \cdot T_4 \quad \mu A_2 = P_1 \cdot IR_2 \cdot T_4$$

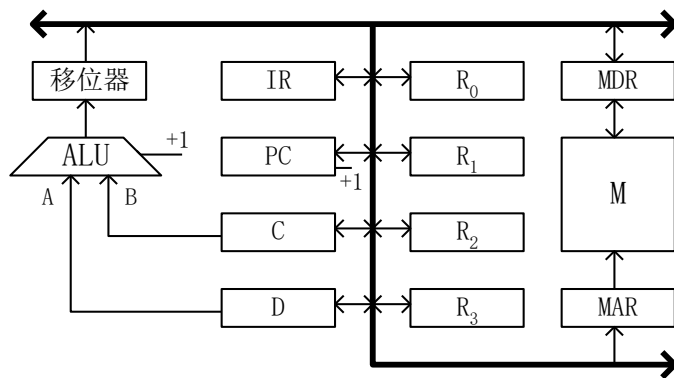
$$\mu A_1 = P_1 \cdot IR_1 \cdot T_4 \quad \mu A_0 = P_1 \cdot IR_0 \cdot T_4 + P_2 \cdot C \cdot T_4$$

其中 μA_5 、 μA_4 、 μA_3 与 μA_2 类似, μA_3 的不同之处是与非门输入为 IR_3 , 而 μA_4 的与非门输入为 P_3 和 IR_4 , μA_5 的与非门输入为 P_3 和 IR_5 。

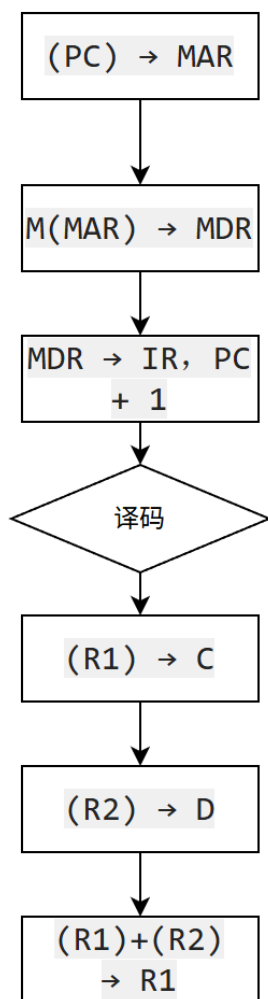


10.

(1) 将 C, D 两个暂存器直接接到 ALU 的 A, B 两个输入端上, 将所有寄存器都双向接到单总线上, 如图所示。



(2) ADD R₁, R₂ 指令的指令周期流程图如图所示。

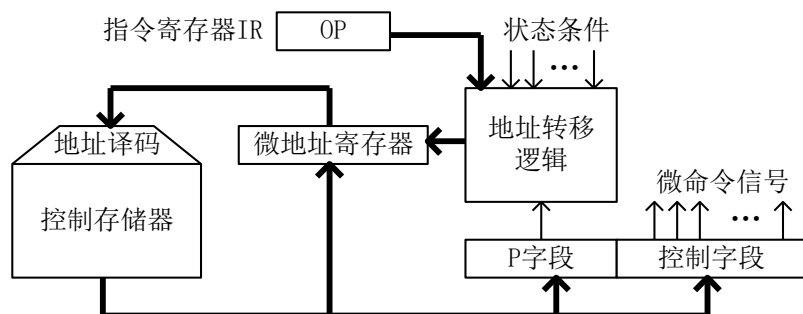


11.

(1) 水平型微指令由操作控制字段、判断测试字段和下地址字段三部分构成。后续微指令地址采用断定方式，根据控制存储器容量（512×48 位）定为 9 位。判别测试字段中每一位作为一个判别标志，那么由于有 4 个转移条件，故该字段为 4 位，则操作控制字段为 48-9-4=35 位。

(2) 微程序控制器逻辑框图如图所示，控制存储器存放实现全部指令系统的微程序，字长

即为微指令字长。微地址寄存器用来存放由控制存储器读出的一条微指令信息，即下地址。微命令寄存器则保存一条微指令的操作控制字段和判别测试字段。地址转移逻辑的输入是指令寄存器的 OP 码、各种状态条件以及判别测试字段所给的判别标志（某一位为 1），其输出用来修改微地址寄存器的适当位数，从而实现微程序的分支转移。



12.

(1) 流水线的操作周期应按各步操作的最大时间来考虑，即流水线时钟周期为 100ns。

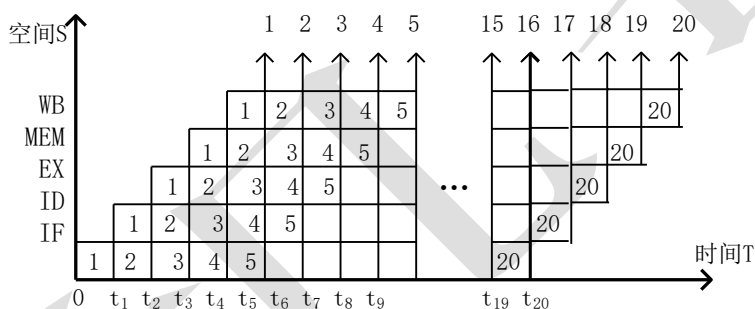
(2) 遇到数据相关时，将相关指令延迟到所需操作数被写回到寄存器后再执行的方式，因此第 2 条指令的

执行至少需要延迟 2 个时钟周期，即 200ns。

(3) 如果设置专用通路技术，则取数之前结果已经计算出来，即至少需要延迟一个时钟周期 100ns。

13.

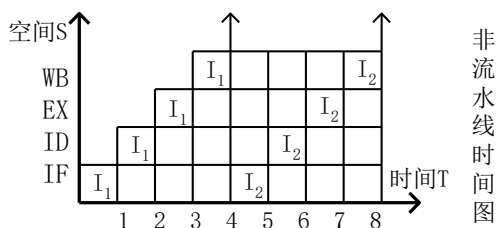
(1) 若流水操作周期为 100ns 每条指令都包含五个过程段。如图所示。

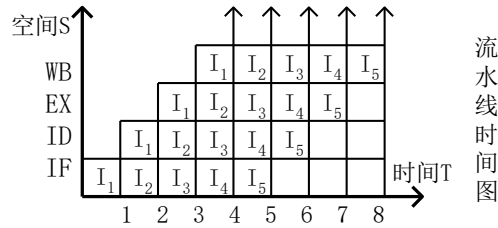


(2) 流水线实际吞吐率：20 条 / 2400ns $\approx$ 833.33 万条指令 / 秒。

(3) 流水线各段时间均为 $\Delta t = 100 \text{ ns}$ ，则完成 20 条指令在 5 段流水线上共需 $T_1 = 5 \times 100 + (20 - 1) \times 100 = 2400 \text{ ns}$ ，而在等效的非流水线上所需时间为 $T_2 = 5 \times 20 \times 100 = 10000$ ，则流水线加速比 $S = T_2/T_1 = 10000/2400 \approx 4.17$

14.





以上分别为执行相同的指令，在 8 个单位时间内，流水计算机完成 5 条指令，而非流水计算机只完成 2 条，显然，流水计算机比非流水计算机有更高的吞吐率。

15. 证明：设流水线各段时间为 t ，则完成 n 条指令在 m 段流水线上共需 $T_1 = m \times t + (n - 1) \times t$ ，则流水线上吞吐量为 $T_1 p = n / T_1$ ，而在等效的非流水线上所需时间为 $T_2 = n \times m \times t$ ，非流水线上吞吐量为 $T_2 p = 1 / (m \times t)$ 。则

$$\frac{T_1 p}{T_2 p} = \frac{T_2}{T_1} = \frac{mnt}{mt + (n - 1)t} = \frac{mn}{m + n - 1} = \frac{m}{\frac{m - 1}{n} + 1}$$

当 n 无限大时，

$$\frac{T_1 p}{T_2 p} = m$$

当 $n = 1$ 时，

$$\frac{T_1 p}{T_2 p} = 1$$

则在 $n > 1$ 时，流水线有更高吞吐率。

16. (1) 写后读 (RAW): 在指令 LDA 写入寄存器 R1 前，指令 ADD 就读出该寄存器内容。

(2) 读后写 (WAR): 在指令 ADD 读出寄存器 R4 前，指令 MUL 就已经写入。

(3) 写后写 (WAW): 在指令 LDA 写入寄存器 R6 前，指令 MUL 就已经写入。

17.(1)

译码段		执行段			写回段	
I ₁	I ₂					
	I ₂	I ₁				
I ₃	I ₄		I ₂		I ₁	
I ₅	I ₆		I ₂ I ₄	I ₃		
	I ₆	I ₅	I ₄	I ₃		I ₂
			I ₆	I ₃		
			I ₆		I ₃	
						I ₄
					I ₅	
						I ₆

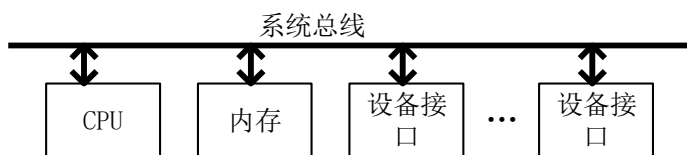
取/存 加法器 乘法器

(2)

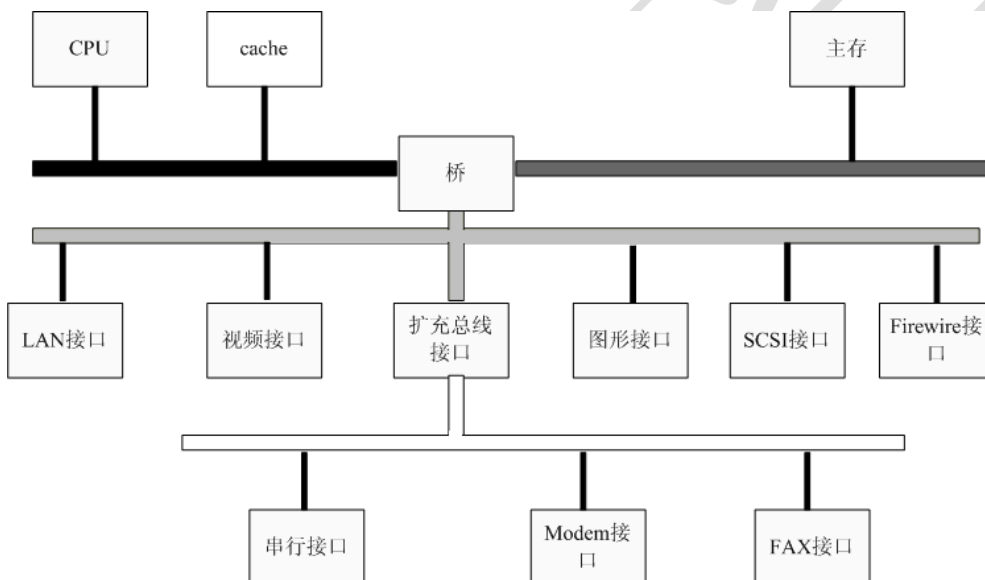
I ₁	F	D	E	W					
I ₂	F	D		E	E	W			
	I ₃	F		D	E	E	E	W	
	I ₄	F		D	E	E			W
			I ₅	F	D	E			W
			I ₆	F	D		E	E	
									W

第六章

1. 单总线结构：它是一组总线连接整个计算机系统的各大功能部件，各大部件之间的所有的信息传送都通过这组总线。其结构如图所示。单总线的优点是允许 I/O 设备之间或 I/O 设备与内存之间直接交换信息，只需 CPU 分配总线使用权，不需要 CPU 干预信息的交换。所以总线资源是由各大功能部件分时共享的。单总线的缺点是由于全部系统部件都连接在一组总线上，所以总线的负载很重，可能使其容量达到饱和甚至不能胜任的程度。故多为小型机和微型机采用。

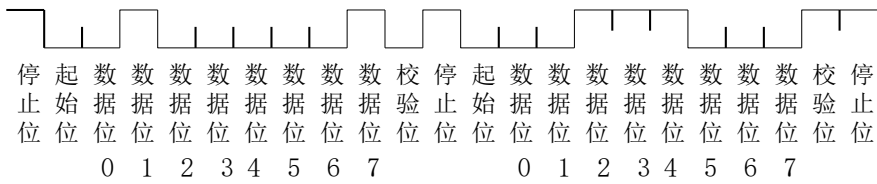


多总线结构：多总线系统结构是通过桥，CPU 总线，系统总线 and 高速总线彼此相连，各大部件的信息传送不是只通过系统总线；体现了高速，中速，低速设备连接到不同的总线上同时进行工作，以提高总线的效率和吞吐量，而且处理器结构的变化不影响高速总线。

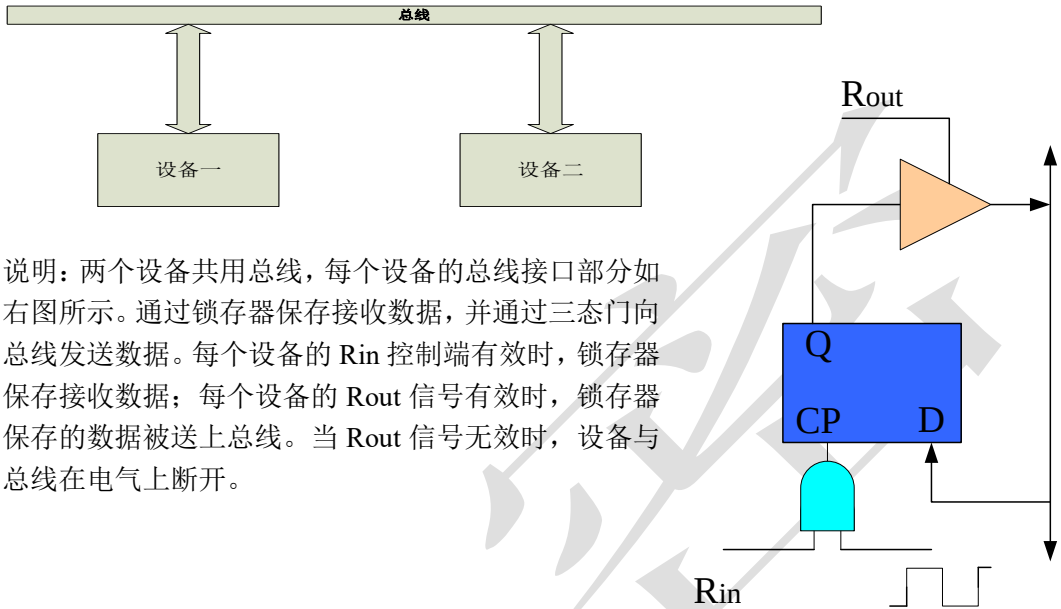


2. (1) 简化了硬件的设计。从硬件的角度看，面向总线是由总线接口代替了专门的 I/O 接口，由总线规范给出了传输线和信号的规定，并对存储器、I/O 设备和 CPU 如何挂在总线上都作了具体的规定，所以，面向总线的微型计算机设计只要按照这些规定制作 CPU 插件、存储器插件以及 I/O 插件等，将它们连入总线即可工作，而不必考虑总线的详细操作。
- (2) 简化了系统结构。整个系统结构清晰，连线少，底板连线可以印刷化。
- (3) 系统扩充性好。一是规模扩充，二是功能扩充。规模扩充仅仅需要多插一些同类型的插件；功能扩充仅仅需要按总线标准设计一些新插件。插件插入机器的位置往往没有严格的限制。这就使系统扩充既简单又快速可靠，而且也便于查错。
- (4) 系统更新性能好。因为 CPU、存储器、I/O 接口等都是按总线规约挂到总线上的，因而只要总线设计恰当，可以随时随着处理器芯片以及其他有关芯片的进展设计新的插件，新的插件插到底板上对系统进行更新，而这种更新只需更新需要更新的插件，其他插件和底板连线一般不需更改。
3. “A”的 ASCII 码为 41H = 01000001B，1 的个数为偶数，故校验位为 0；“8”的 ASCII 码为

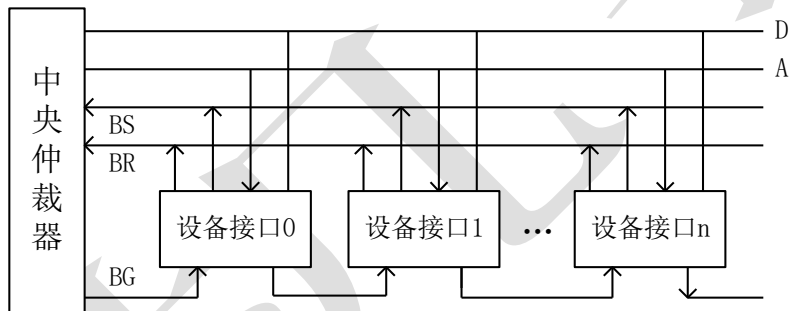
38H = 00111000B, 1 的个数为奇数, 故校验位为 1。



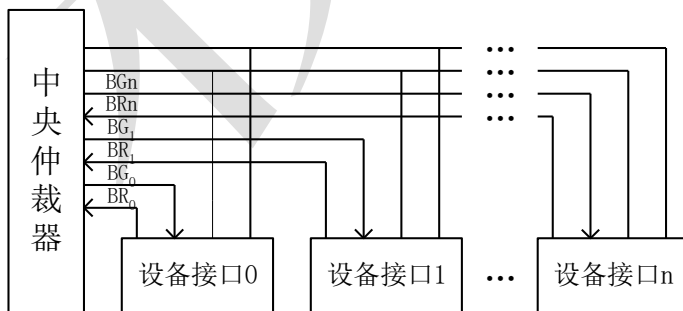
4. 逻辑图如下:



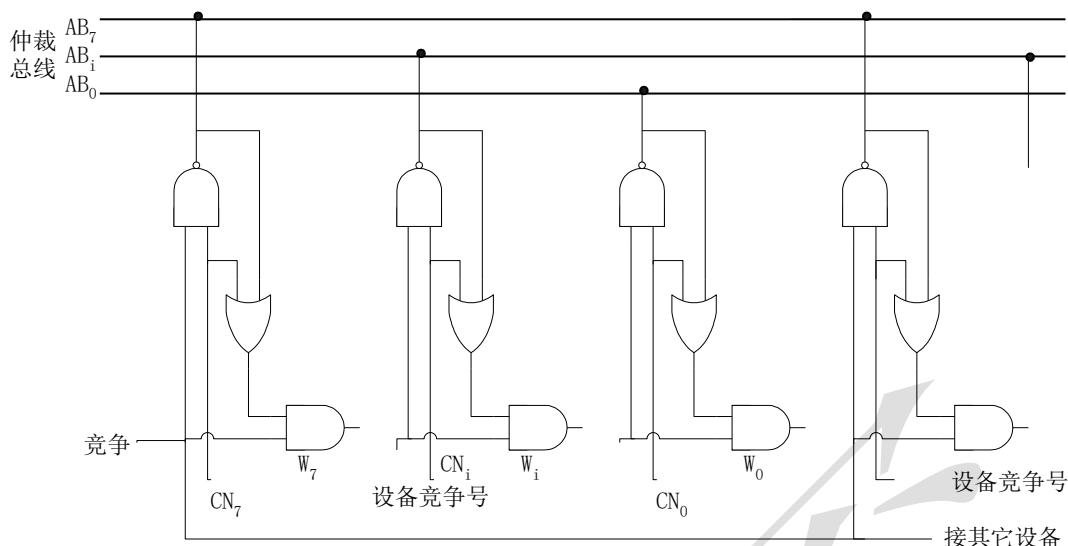
5.



6.



7.



8. C

【解析】同步通信中，接收端和发送端的收发时钟是严格同步的，即通信过程中采用同一个公共时钟信号进行同步。由于收发时钟是严格同步的，在接收端就不会因接收时钟和发送时钟不一致所造成的时钟误差积累。正是这种收发时钟的严格同步，才允许在同步通信中，可以用很高的传输速率进行数据传输。

异步通信中，接收端和发送端的收发时钟是不需要严格同步的，即通信过程中收发时钟是异步的，正是这种收发时钟的异步，在接收端就会因接收时钟和发送时钟不一致所造成的时钟误差积累，这就使得异步通信的传输速率相对较低。

综上所述，同步通信传输频率高是因为用公共时钟进行同步，C 项正确，其他三项并不是造成同步异步传输频率不同的原因。

9. BAC

【解析】链式查询方式的缺点是对询问链的电路故障很敏感。独立请求方式的优点是响应时间快。计数器定时查询方式中，每次计数从“0”开始，各设备的优先次序与链式查询法相同，优先级固定。每次从中止点开始，每个设备使用总线的优先级相等。

10. A

【解析】根据波特率和传送数据的位数，计算字符传送速率=9600/(7+1+1+1)=960 字符/秒。

11. D

【解析】系统总线中的地址线不仅用来传送主存的地址，同时也可传送 I/O 设备接口电路的地址。

12. A

【解析】控制线用来指明数据传送的方向（存储器读、存储器写、I/O 读、I/O 写）。而数据信息由数据总线来完成，时序信号由时序产生电路完成。

13. 存储总线周期用于对内存读写，I/O 总线周期对接口中的端口进行读写。

14. DCAB

15. ABDC

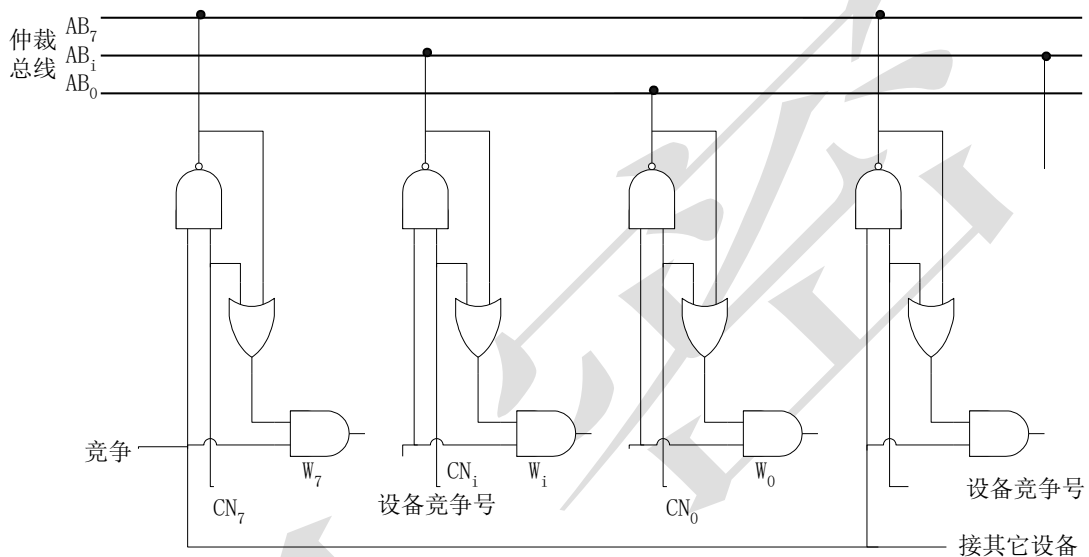
16. 由典型的多总线结构框图可知：

(1) PCI 总线体系结构中的三种桥分别为：HOST 桥、PCI/LAGACY 总线桥、PCI/PCI 桥。

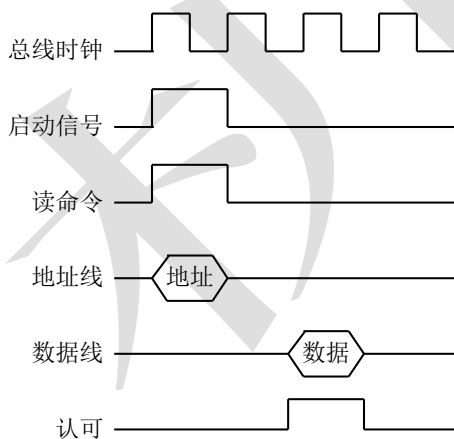
(2) 桥在 PCI 总线结构中起着重要作用，它连接两条总线，使彼此间相互通信。桥又是

一个总线转换部件，可以把一条总线的地址空间映射到另一条总线的地址空间上，从而使系统中的任意一个总线主设备都能看到同样的一份地址表。桥可以实现总线间的猝发式传送，可使所有的存取都按 CPU 的需要出现在总线上。由上可见，以桥连接实现的 PCI 总线结构具有很好的扩充性和兼容性，允许多条总线并行工作。

17. 分布式仲裁不需要中央仲裁器，每个潜在的主方功能模块都有自己的仲裁号和仲裁器。当它们有总线请求时，把它们唯一的仲裁号发送到共享的仲裁总线上，每个仲裁器将仲裁总线上得到的号与自己的号进行比较。如果仲裁总线上的号大，则它的总线请求不予响应，并撤消它的仲裁号。最后，获胜者的仲裁号保留在仲裁总线上，分布式仲裁是以优先级仲裁策略为基础。



18. 总线的一次信息传送过程，大致可分为：请求总线，总线仲裁，寻址，信息传送，状态返回。



19. 设总线带宽用 D_r 表示，总线时钟周期用 $T = 1/f$ 表示，一个总线周期传送的数据量用 D 表示，根据定义可得：

$$D_r = T / D = 8B \times 70MHz = 560MB/s$$

20. (1) PCI 总线: 基于并行共享总线，采用同步定时协议和集中式仲裁策略，支持自动配置。32 位/33 MHz 版本理论带宽为 133 MB/s，64 位/66 MHz 版本可达 533 MB/s（共享带宽）。适用于低成本小型系统（如早期声卡、低速网卡），受限于并行信号同步问题（如时钟偏移）和低扩展性。

(2)PCIe 标准:基于串行点对点架构,采用全双工差分信号传输,支持数据包协议(TLP/DLP)和独立通道(Lane)。单通道双向带宽逐代倍增(PCIe 5.0 单通道达 8 GB/s),多通道叠加(如 x16 总带宽 64 GB/s)。满足高性能需求(如 GPU、NVMe SSD、高速网卡),支持低延迟、高扩展性(通过交换器灵活扩展)和高级功能(如 MSI 中断、动态电源管理)。

21.鲲鹏 920 处理器片上系统通过多层次的互联机制实现高效的片内与片间通信。

在片内层面,其采用分布式网络片上网络(NoC)架构,通过低延迟链路和分层路由策略连接多核 CPU、共享缓存、内存控制器及专用加速模块,确保核心间的高吞吐量数据传输与动态负载均衡。为保障多核协同运算的一致性,系统集成了基于目录的缓存一致性协议(如 MESI/MOESI),有效管理共享 L3 缓存的数据同步,减少核心间通信开销。关键功能模块(如 DDR 控制器、PCIe 控制器)则通过高速 AMBA AXI 总线和交叉开关互联,提供高带宽、低延迟的片内数据传输通道。

在片间扩展方面,鲲鹏 920 支持多处理器一致性互联,利用高速串行链路(如基于 PCIe 5.0 物理层的自定义协议)连接多个处理器芯片,实现跨芯片的全局内存共享与缓存一致性,适配 NUMA 架构的大规模计算需求。同时,其集成 PCIe 4.0/5.0 控制器,通过多通道 x16 配置为 GPU、NVMe SSD 等外设提供高达 64 GB/s 的双向带宽,并支 100/200GbE 网络接口与 RDMA 技术,实现跨节点低延迟数据传输,满足分布式存储与计算场景。物理层采用差分信号传输与先进封装技术(如 2.5D/3D 堆叠),优化信号完整性并降低功耗;协议层则融合内存语义通信(如 CXL 协议)与异构计算框架,减少数据搬移开销,进一步提升整体系统效率。

通过上述多层次的互联设计,鲲鹏 920 在片上多核协同、片间扩展及异构资源整合上实现了高性能与高可靠性的平衡,适用于云计算、数据中心等对带宽、延迟及一致性要求严苛的场景。

第七章

1. D

2. C、D、C、A

$$3. \frac{1024 \times 1024 \times 256}{8 \times 8} = 1MB$$

$$4. \text{格式化容量} = \text{扇区容量} \times \text{每道扇区数} \times \text{磁道总数} \\ = 512 \times 9 \times 100 \times 2 = 921600B$$

5. 设读写一块信息所需总时间为 t_B , 平均找道时间为 t_s , 平均等待时间为 t_t , 读写一块信息的传输时间为 t_m , 则 $t_B = t_s + t_t + t_m$. 假设磁盘以每秒 r 转速率旋转, 每条磁道容量为 N 个字, 则数据传输率 $= rN$ 个字/秒。

又假设每块的字数为 n , 因而一旦读写头定位在该块始端, 就能在 $t_m \approx (n/rN)$ 秒的时间中传输完毕。 t_t 是磁盘旋转半周的时间, $t_t = (1/2r)$ 秒。由此可得:

$$t_B = t_s + \frac{1}{2r} + \frac{n}{rN}$$

6. 数据传输率 $Dr = \text{磁盘转速 } r \text{ (转/秒)} \times \text{每条磁道容量 } N \text{ (字节)},$

$$\text{则 } \frac{185000B/s}{4000 \text{ 转}/60s} = 2775B/\text{转} = 2775B/\text{道}$$

所以磁盘容量为 $2 \times 220 \times 2775B \approx 1.16MB$

7. (1) $275 \times 12288 \times 4 = 12.89MB$

(2) 最高位密度 $D1$ 按最小磁道半径 $R1$ 计算 ($R1 = 115mm$):

$$D1 = 12288 \text{ 字节} / 2\pi R1 = 17 \text{ 字节} / mm$$

最低位密度 $D2$ 按最大磁道半径 $R2$ 计算:

$$R2 = R1 + (275 \div 5) = 115 + 55 = 170mm$$

$$D2 = 12288 \text{ 字节} / 2\pi R2 = 11.5 \text{ 字节} / mm$$

$$(3) \frac{3000}{60} \times 12288 = 600KB/s$$

$$(4) \frac{1}{2} \times \frac{60}{3000} \times 1000 = 10ms$$

(5)

16	15	14	6	5	4	3	0
台号	柱面(磁道)号	盘面(磁头)号	扇区号				

此地址格式表示有 4 台磁盘, 每台有 4 个记录面, 每个记录面最多可容纳 512 个磁道, 每道有 16 个扇区。

8.

存取时间 = 平均查找时间 + 平均等待时间

$$= 60 + \frac{1}{2} \times \frac{60}{2400} \times 1000 = 72.5ms$$

$$Dr = 96 \times \frac{2400}{60} = 480KB/s$$

9. (1) 根据公式数据传输率 $C = D \times V$, 其中 D 为记录位密度, V 为线速度 $D = \frac{C}{V} = \frac{128000 \text{ 字节/秒}}{2m/s} =$

64000 字节/m

(2) 传送一个数据块所需时间为

$$t = \frac{1024 \text{ 字节}}{128000 \text{ 字节/秒}} = \frac{1}{125} \text{ 秒}$$

一个数据块占用长度为

$$l = v * t = 2m/s * \frac{1}{125} s = 0.016m$$

每块间隙 $L = 0.014m$, 数据块总数为

$$\frac{600-4}{l+L} = 19867 \text{ 块}$$

故磁带存储器有效存储容量为

$$19867 \text{ 块} * 1K \text{ 字节} = 19867K \text{ 字节}$$

10. (1) 道密度为 100 道/英寸, 而可供记录的部分宽 5 英寸, 所以磁道数为 $5 \times 100 = 500$ 道; 又因最内圈磁道直径为 $18 - 2 \times 5 = 8$ 英寸, 所以周长为 8π 英寸, 对应道容量为 8000π 位/道, 面容量为 $8000\pi \times 500 = 4000000\pi$ 位/面, 盘片总容量为 $4000000\pi \times 20 = 80000000\pi$ 位 $= 80\pi$ 兆位。

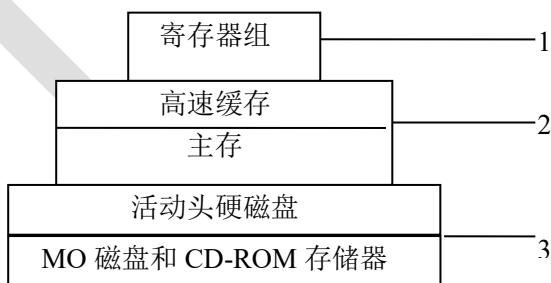
(2) 数据传输率已知, 根据公式数据传输率 = 每道字节数 $\times$ 每秒转速, 可求得每分钟转速 = 数据传输率 $\times 60$ / 每道字节数 $= 1024 \times 1024 \times 60 / (1000\pi) \approx 20026$ 转/分。

11. (1) $[(30 \times 10^{-3} + 10 \times 10^{-3} + 3000/500 \times 10^{-3}) \times 2 + 4 \times 10^{-3}] \times 1000 = 96s$

(2) $[(30 \times 10^{-3} + 5 \times 10^{-3} + 3000/1000 \times 10^{-3}) \times 2 + 4 \times 10^{-3}] \times 1000 = 80s$

12. (1) 存储容量从大到小依次为: 活动头磁盘存储器, MO 磁盘, CD-ROM 存储器, 主存, 高速缓存, 寄存器组; 按存储周期从大到小依次为: CD-ROM 存储器, MO 磁盘, 活动头磁盘存储器, 主存, 高速缓存, 寄存器组。

(2) 可构成如下的多级存储体系:



(3) CPU 和高速缓存以及 CPU 和主存之间有直接的数据通路, 而 CPU 与外存之间不存在直接的数据通路, CPU 访问硬盘和光盘时都需要先将信息调入主存。

13. 刷新存储器用于存储一帧图像信息的存储器, 字符显示方式下存放待显示字符的 ASCII 码和属性码, 图形显示方式下存放每个像素的颜色值, 其容量与显示器的分辨率和颜色设置有关。若 CRT 的分辨率为 1024×1024 个像素点, 颜色深度为 24 位, 则刷新存储器的存储容量为 $1024 \times 1024 \times 3B = 3MB$

14. (1) 颜色深度 24 位, 使用 3 个字节表示, 故刷新存储器的存储容量是: $1024 \times 768 \times 3 = 2.25MB$ 。

(2) 刷新存储器的带宽 =

$$\text{分辨率} \times \text{每个像素点颜色深度} \times \text{刷新速度} = 1024 \times 768 \times 3 \times 72 = 165888KB/s = 162MB/s$$

15. (1) 存储原理与结构: 机械硬盘依赖机械部件 (盘片、磁头、马达等) 实现数据读写。盘片高速旋转 (通常每分钟几千转), 磁头通过电磁流改变盘片磁性区域的极性来记录数据。其结构精密, 但易受震动、灰尘影响, 需完全密封, 而固态硬盘基于闪存芯片 (如 NAND) 存

储数据，无机械部件。通过改变电荷状态实现数据读写，抗震性极强，适合移动设备；性能表现:机械硬盘的读写速度约为 120-250MB/s，受限电机转速和寻道时间；而固态硬盘的 SATA 协议型号可达 500MB/s，NVMe 协议（如 PCIe 4.0）甚至高达 3500-7500MB/s，且随机读写速度远超机械硬盘（延迟低至 0.1 毫秒）。同时，固态硬盘功耗更低（无电机），且无噪音；机械硬盘因马达转动会产生噪音和发热。固态硬盘在碰撞、倾斜环境下更稳定，但其闪存芯片有擦写次数限制，机械硬盘则易因震动导致磁头划伤盘片，理论上寿命更长（无写入次数限制）。

(2) 如健康监测与预警、固件更新与优化、数据安全等功能。

第八章

1.A、B、C

2.C

3.A

4.C

5.组织外围设备和内存进行数据传输；控制外围设备；选择型；多路型。

6.能响应，因为设备 A 的优先级比设备 B 高。若要设备 B 总能立即得到服务，可将设备 B 从第二级取出来，单独放在第三级上，使第三级的优先级最高，即令 $IM_3 = 0$ 。

7.依次处理设备 A，设备 D，设备 G 的时间为：

$$T_1 = t_1 + t_2 + t_3 + t_4 + t_A$$

$$T_2 = t_1 + t_2 + t_3 + t_4 + t_D$$

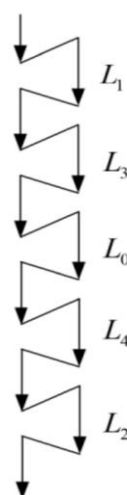
$$T_3 = t_1 + t_2 + t_3 + t_4 + t_G$$

$$\text{总时间为 } T = T_1 + T_2 + T_3 = 3 * (t_1 + t_2 + t_3 + t_4) + t_A + t_D + t_G$$

8.(1)

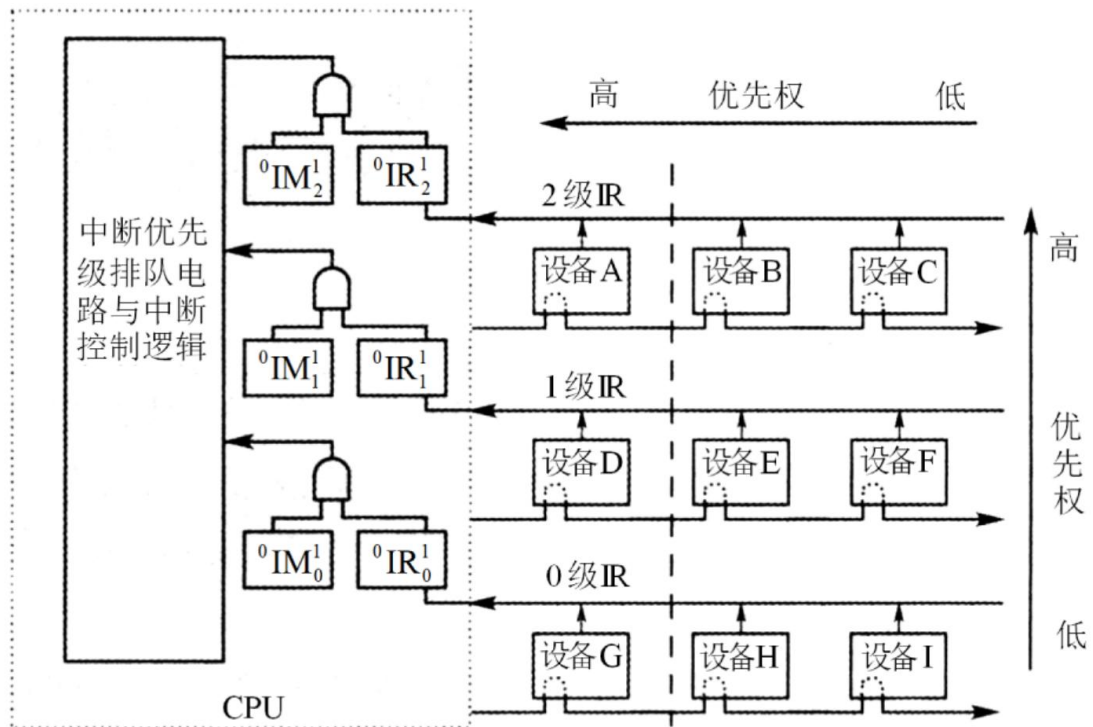
中断处理程序	中断处理级屏蔽位				
	L_0	L_1	L_2	L_3	L_4
L_0	1	0	1	0	1
L_1	1	1	1	1	1
L_2	0	0	1	0	0
L_3	1	0	1	1	1
L_4	0	0	1	0	1

(2) 5 级中断同时发出中断请求时的过程示意图如下图

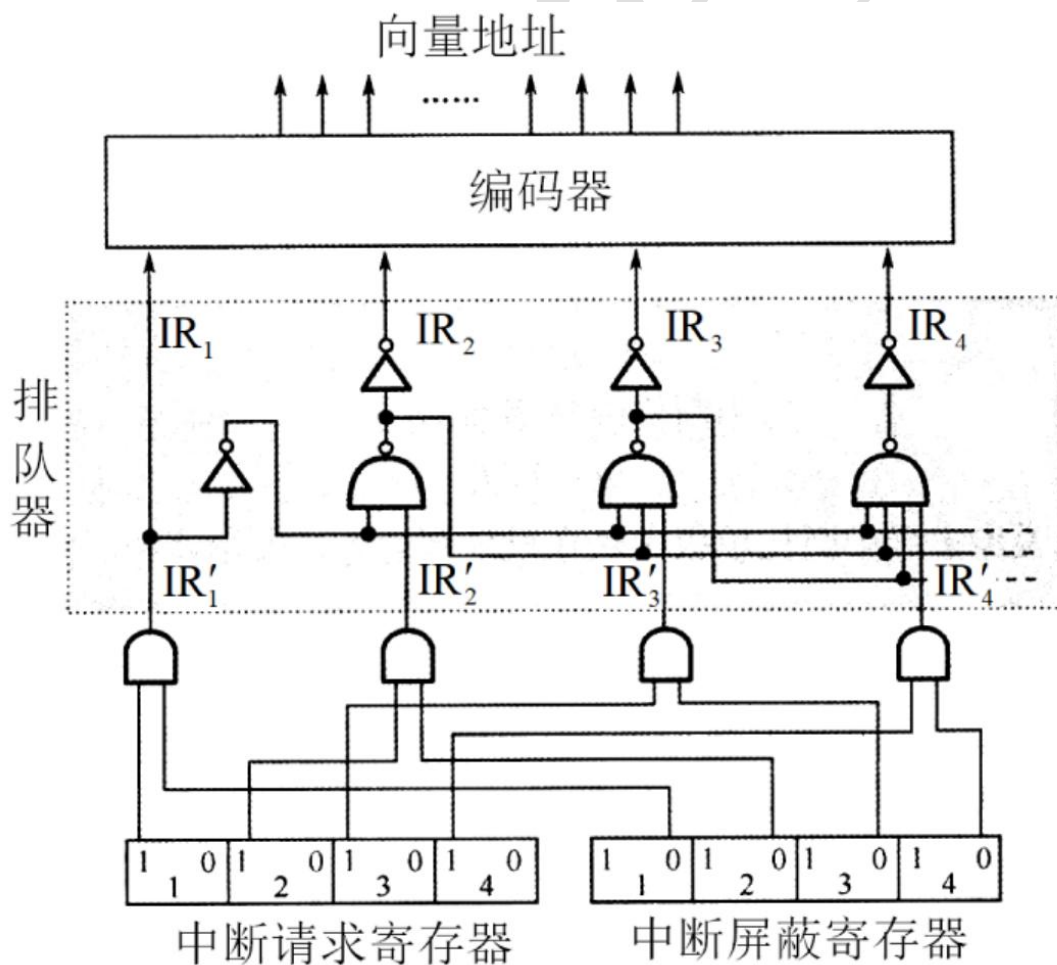


9.要将通用寄存器内容保存到主存中去。只需保存中断处理程序用到的那 2 个寄存器内容。

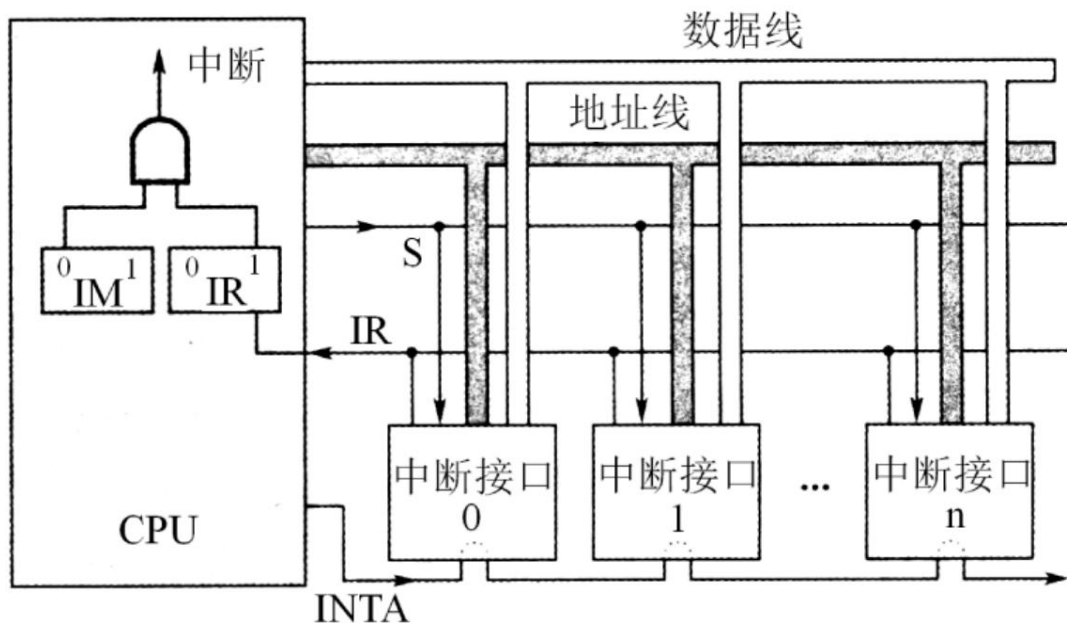
10. 二维中断结构判优逻辑总图如下



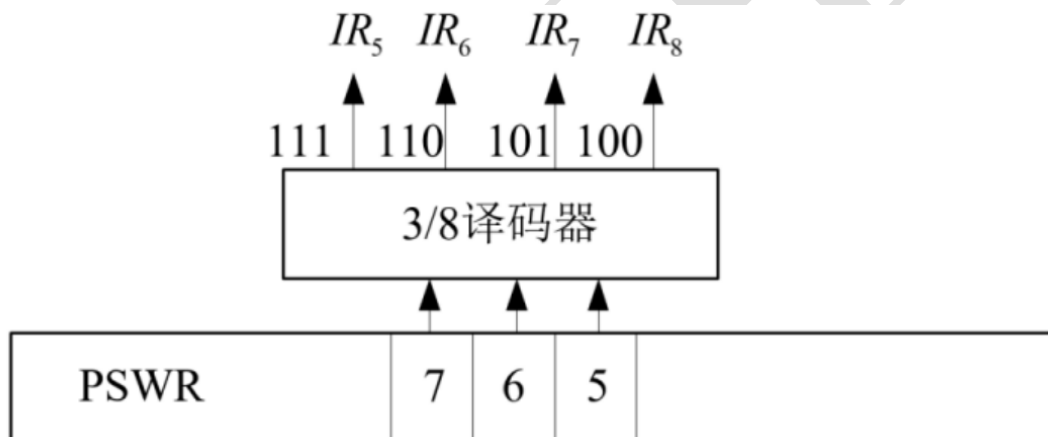
主优先级判定电路如下



次优先级判定电路如下



将 CPU 程序的四级加在主优先级判定电路中，即在排队器中依次添加 IR_5, IR_6, IR_7, IR_8 在中断屏蔽器中依次加上四位标记位，而请求寄存器 PSWR 中 CPU 程序优先级逻辑电路如下。



11.(1) $IM_2=IM_1=IM_0=011$

(2) $IM_2=IM_1=IM_0=001$

(3) 若要设备 B 总能立即得到服务，可将设备 B 从第二级取出来，单独放在第三级上，使第三级的优先级最高，即令 $IM_3=0$ 。

12.D

【解析】A 项，DMA 和 CPU 可并行执行；B 项，中断发生时，CPU 首先要根据中断优先级判定是否响应中断，响应中断后才会有保存现场操作；C 项，DMA 的数据传送是以数据块为基本单位进行的，每传送一个数据块窃取一个指令周期。

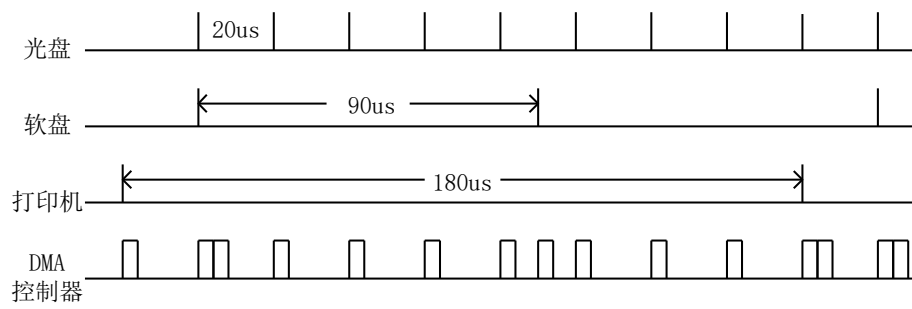
13. 外部中断、可屏蔽中断、非屏蔽中断、内部中断、异常、执行软件中断指令

14.(1) 通道方式：可以实现对外设的统一管理和外设与内存之间的数据传送，大大提高了 CPU 的工作效率。

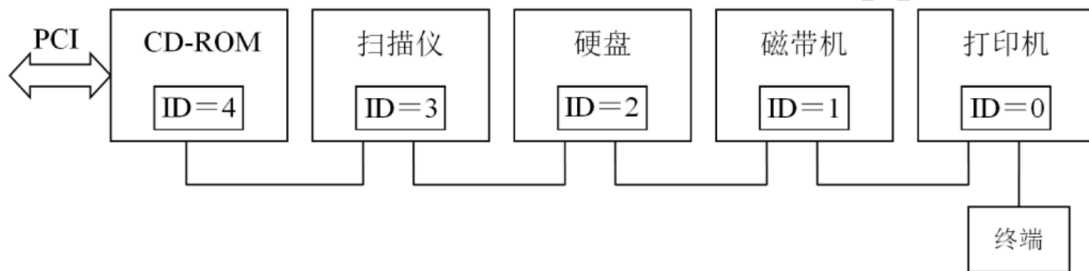
(2) DMA 方式：数据传送速度很高，传送速率仅受到内存访问时间的限制。需要更多硬件，适用于内存和高速外设之间大批数据交换的场合。

(3) 中断方式：一般适用于随机出现的服务，且一旦提出要求应立即进行，节省了 CPU 的时间开销，但硬件结构稍复杂一些。

15. 多路 DMA 控制器的工作时空图如下所示



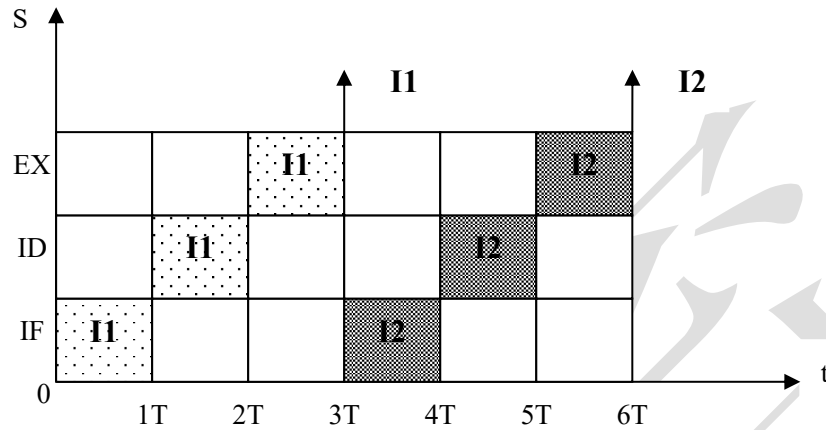
16. 每个 SCSI 设备有自己唯一的设备号 ID=0~4, 其中 ID=4 的设备具有最高优先级, 如下所示。



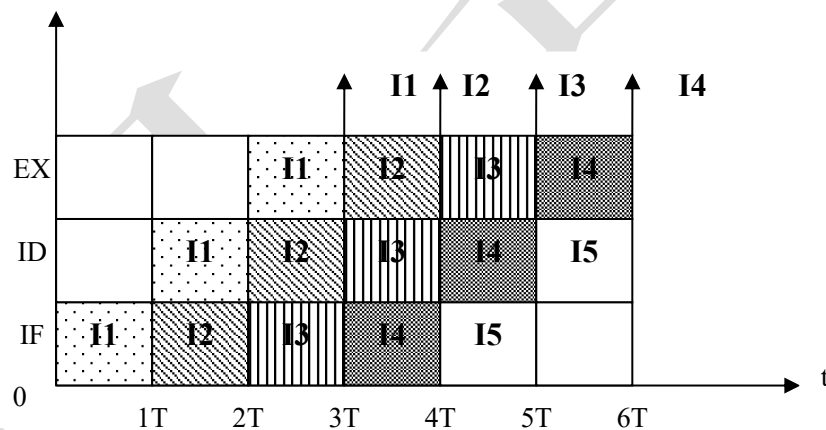
第九章

1. 略

2. (1) 设三个子过程为取指令 (IF)、指令译码 (ID)、指令执行 (EX)，则指令顺序执行和流水执行方式时空图如下图 (a) 和 (b) 所示。



(a) 顺序执行时空图



(b) 流水执行时空图

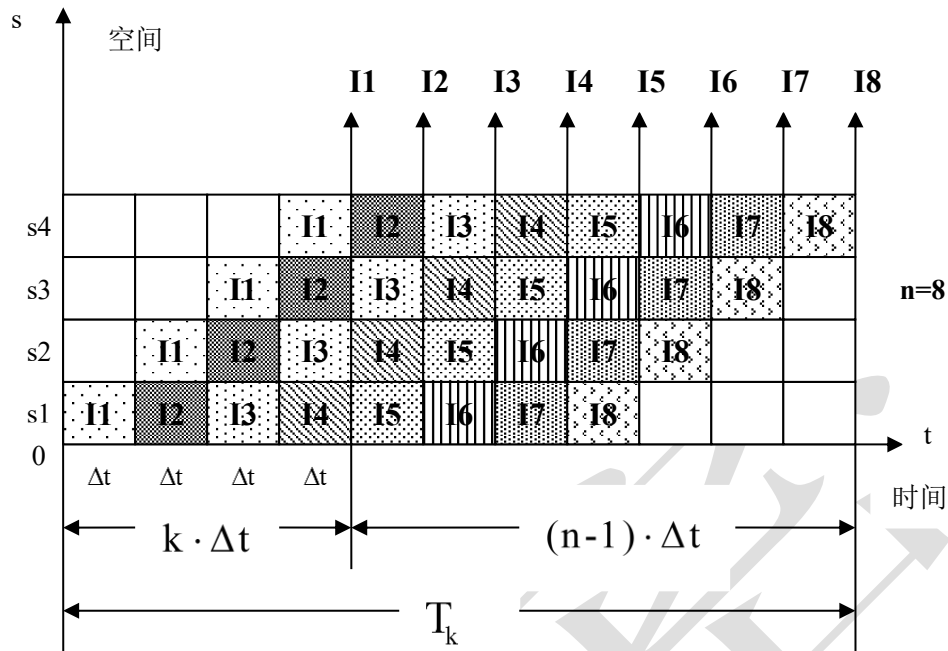
(2) 顺序执行方式: $n = 1000$ 条, $T = 100\text{ns}$

总时间: $t_1 = 3 \times n \times T = 3 \times 1000 \times 100 = 300000\text{ns}$

流水执行方式: $t_2 = (n + 2)T = 1002 \times 100 = 100200\text{ns}$

(3) 加速比: $Se = t_1 / t_2 = 300000 / 100200 \approx 2.999$ 倍

3. (1) 设 $k=4$ 个, 流水段为取指 (s1)、译码 (s2)、执行 (s3)、存结果 (s4)。



(2)从上图看出, 用 $k=4$ 个时钟周期 (Δt) 完成第 1 条指令, 其余 $n-1$ 个时钟周期完成 $n-1$ 条指令。因此流水线完成 n 条指令所需的总时间为

$$T_k = (k + n - 1)\Delta t$$

根据定义, 吞吐率 P 为

$$P = \frac{n}{k} = \frac{n}{(k+n-1)\Delta t}$$

(3)顺序执行 n 条指令所用的总时间 T_0 为

$$T_0 = (k \cdot \Delta t) \cdot n$$

根据定义, 加速比的公式为

$$S_e = \frac{T_0}{T_k} = \frac{nk\Delta t}{(k+n-1)\Delta t} = \frac{nk}{k+n-1}$$

4.D

【解析】超线程技术是同时多线程技术在英特尔处理机上的具体实现。

5.C

【解析】总线共享 cache 优点是结构简单、易于设计实现、通信速度快, 但缺点是总线结构的可扩展性较差, 只适用于核心数较少的情况。

6.C

【解析】存储墙是提升计算速度的第一难题, 它指由于处理器和 DRAM 按照不同的指数函数增长, 当达到某一程度时, 处理器速度的增长对系统性能的影响会被慢速的 DRAM 所屏蔽。

7. 设总指令数为 m , 并行指令数为 $m(P)$, 顺序指令数为 $m(S)$, 则总执行时间 T 为:

$$T = \frac{m(P)}{nx} + \frac{m(S)}{x} = \frac{mF}{nx} + \frac{m(1-F)}{x}$$

有效 MIPS 表达式为:

$$MIPS = \frac{m}{T} = \frac{m}{\frac{mF}{nx} + \frac{m(1-F)}{x}} = \frac{m}{\frac{mF + nm - nmF}{nx}} = \frac{nx}{n(1-F) + F}$$

8. 在上式中代入已知条件:

$$64 = \frac{32 \times 8}{32(1-F) + F}$$

求得 $F \approx 90.32\%$ 。

9. 设加速比为 k , 可加速部分比例为 F_e , 理论加速比为 S_e , 根据 Amdahl 定律:

$$k = \frac{1}{(1-F_e) + F_e/S_e}$$

为了简单化, 假设程序只在两种模式下运作: (1) 使用所有处理机的运行模式; (2) 只用一个处理机的串行模式。假设并行模式下的理论加速比 S_e 即为多处理机的台数, 加速部分的比例 F_e 即并行部分所占的比例, 代入上式有:

$$80 = \frac{1}{(1-F_e) + F_e/100}$$

求得并行比例 $F_e = 0.9975 = 99.75\%$, 串行比例 $1-F_e = 0.25\%$

第十章

1. 电子计算机和量子计算机的特点如下表所示:

比较内容	电子计算机	量子计算机
物理原理	半导体电子机制	量子力学
存储、计算基本单位	经典比特 (bit)	量子比特 (qbit)
电路	电子电路、电子逻辑门	量子电路、量子逻辑门
计算方式	单个单元内是串行计算	单个单元也是并行计算
内部信息存储状态	确定的 0,1 序列	不确定, 只是一种统计概率
存储单元信息读取	可以多次重复读取	只能测量一次, 并导致状态坍缩

量子计算机的优点包括:

- (1) 运行速度较快;
- (2) 存储能力较强;
- (3) 信息处理能力较强;
- (4) 可以模拟各种物理系统。

目前技术上还存在的难题主要有:

- (1) 曾经所有的量子计算机模型都是把量子计算机看成是一个不与外界环境发生作用的孤立系统, 而不是实际模型。
- (2) 存在许多不利于实现量子计算机的制约因素, 例如去相干、激光脉冲不可控制的变化、热噪声等等。
- (3) 量子计算机仍然易出错, 而且不易纠错。

2. 设量子态矢量为 $|\nu\rangle$:

- (1) $|\nu\rangle = [0 \ 1 \ 1]^T = |011\rangle$
- (2) 由 $0.52=0.25$ 得 $|\nu\rangle = 0.5|00\rangle + 0.5|01\rangle + 0.5|10\rangle + 0.5|11\rangle$
- (3) $|\nu\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle$

3. 根据题目描述:

- (1) 基态 $|00\rangle$ 就是二进制数 0, 因此出现概率是 100%。
- (2) 基态 $|00\rangle$ 对应的复数系数为 0, 因此出现概率是 0%。
- (3) 由于 $|a|^2 + 0.42 + 0.22 + 0.52 = 1$, 因此 $|00\rangle$ 出现概率
 $|a|^2 = 1 - 0.4^2 + 0.2^2 + 0.5^2 = 0.55 = 55\%$ 。

4. 根据题目描述:

- (1) 100%测得 0101。
- (2) $0.4^2=0.16$, $0.2^2=0.04$, $0.8^2=0.64$, 则
16%测得 00, 16%测得 01, 4%测得 10, 64%测得 11。
- (3) $|0.5 + 0.5i|^2=0.5$, $|0.5 - 0.5i|^2=0.5$, 则
50%测得 010, 50%测得 111。

5. 设量子比特为 $|\nu\rangle = a_0|0\rangle + a_1|1\rangle$, 则非门的酉变换结果

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} (a_0|0\rangle + a_1|1\rangle) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} a_1 \\ a_0 \end{bmatrix} = a_1|0\rangle + a_0|1\rangle, \text{ 由此}$$

$$(1) \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} |0\rangle = |1\rangle$$

$$(2) \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} (0.6|0\rangle + 0.8|1\rangle) = 0.8|0\rangle + 0.6|1\rangle$$

$$(3) \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} ((0.5+0.5i)|0\rangle + (0.5-0.5i)|1\rangle) = (0.5-0.5i)|0\rangle + (0.5+0.5i)|1\rangle$$

6. 量子比特依次经过 Z 门、H 门、X 门之后变为（矢量与变换矩阵是从右到左运算）

$$\begin{aligned} |v\rangle &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} (a_0|0\rangle + a_1|1\rangle) \\ &= \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} \\ &= \begin{bmatrix} \frac{a_0 + a_1}{\sqrt{2}} \\ \frac{a_0 - a_1}{\sqrt{2}} \end{bmatrix} = \frac{a_0 + a_1}{\sqrt{2}} |0\rangle + \frac{a_0 - a_1}{\sqrt{2}} |1\rangle \end{aligned}$$

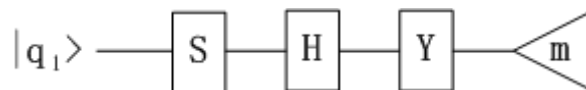
7. （变换矩阵是从右到左运算，与量子电路左右颠倒）

$$(1) \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} -i & -1 \\ i & -1 \end{bmatrix}$$

$$(2) \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$(3) \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -i \\ 0 & 0 & i & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -i \\ 1 & 0 & 0 & 0 \\ 0 & 0 & i & 0 \end{bmatrix}$$

8. 电路图如下所示



9. 光量子技术、离子阱技术、超导量子计算技术、核磁共振技术、超冷原子技术。

10. (1) 穷举法找到正确密钥平均需要尝试 $2^{256}/2 = 2^{255}$ 次，平均耗时 $2^{255} \times 10^{-6} = 5.7896 \times 10^{70}$ 秒，约为 1.8346 亿亿亿亿亿亿年。

(2) Grover 算法平均要经过 $\pi N^{1/2}/4$ 轮酉变换迭代, $N=256$, 平均耗时 $\pi N^{1/2}/4 \times 10^{-6} = \pi 256^{1/2}/4 \times 10^{-6} = 3.1416 \times 16/4 \times 10^{-6} = 1.2566 \times 10^{-6}$ 秒, 约为 1.26 微秒。

11. 算法的时间复杂度是用数量级的不同来衡量的, 成倍增长之中的倍数 (系数 k) 并不能提升数量级, 会被忽略, 例如 $O(kN)=O(N)$ 。所以电子计算机的并行算法相比串行算法的时间复杂度是一样的。

量子计算机的 Shor 算法相比电子计算机的数域筛法, 本质上已经是不同的算法, 时间复杂度的数量级不同。Shor 算法在多项式时间复杂度以内, 意味着素因数分解已经成为 P 类问题, 而数域筛法则超出了多项式时间复杂度, 两种算法的速度差异相比“成倍增长”要显著得多。